

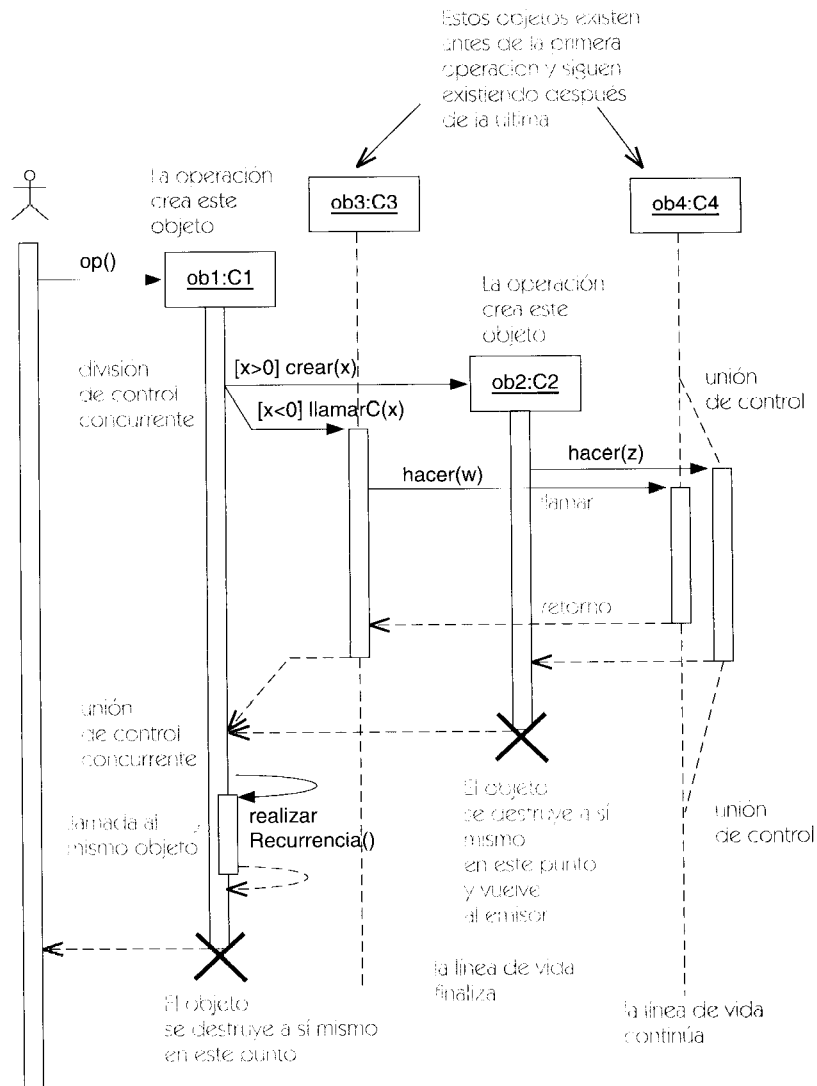


Figura 13.70 Un diagrama de secuencia con flujo de control procedural

invertir los ejes si se desea). Con frecuencia sólo son importantes las secuencias de mensajes pero en aplicaciones de tiempo real el eje temporal puede ser una métrica. La ordenación horizontal de los objetos no tiene ningún significado.

Cada objeto se representa en una columna distinta. Se pone un símbolo de objeto (un rectángulo con el nombre del objeto subrayado) al final de una flecha que representa el mensaje que ha creado el objeto; está situada en el punto vertical que denota el instante en que se crea el objeto. Si un objeto existe desde antes de la primera operación del diagrama, se dibuja el símbolo del objeto en la parte superior del diagrama, antes de todo mensaje. Se dibuja una línea discontinua desde el símbolo de objeto hasta el punto en que se destruye el objeto (si es que esto sucede durante el período de tiempo que muestra el diagrama). Esto se llama línea de vida del objeto. Se pone una X grande en el punto en que deja de existir el objeto o en el punto en que el objeto se destruye a sí mismo. Para cualquier período durante el cual esté activo el

objeto, la línea de vida se amplía para ser una doble línea continua. Esto incluye toda la vida de un objeto activo o las activaciones de un objeto pasivo, un período durante el cual se está ejecutando una operación del objeto, incluyendo el tiempo durante el cual la operación espera al retorno de alguna operación que ella haya invocado. Si el objeto se llama a sí mismo recursivamente, bien sea de manera directa o indirecta, entonces se superpone otra copia de la doble línea continua para mostrar la doble activación (potencialmente, podrían ser más de dos copias). El orden relativo de los objetos no tiene significado, aun cuando resulta útil organizarlos de tal modo que se minimice la distancia que tienen que recorrer las flechas de mensajes. Se puede poner cerca de ella un comentario relativo a la activación.

Cada mensaje se representa mediante una flecha horizontal que va desde la línea de vida del objeto que envió el mensaje hasta la línea de vida del objeto que ha recibido el mensaje. Se puede poner una etiqueta en el margen del lado opuesto a la flecha para denotar el momento en que se envía el mensaje. En muchos modelos se supone que el mensaje es instantáneo, o al menos atómico. Si un mensaje requiere un cierto tiempo para llegar a su destino, entonces la flecha del mensaje se dibuja diagonalmente hacia abajo, de tal modo que el instante de recepción sea posterior al instante de emisión. Los dos extremos pueden tener etiquetas para indicar el momento en que se ha enviado o recibido el mensaje.

Para un flujo de objeto asíncrono entre objetos activos, los objetos se representan mediante líneas dobles continuas y los mensajes se representan como flechas. Se pueden enviar simultáneamente dos mensajes pero no se pueden recibir simultáneamente porque no se puede garantizar una recepción simultánea. La Figura 13.69 muestra un diagrama de secuencia asíncrono.

La Figura 13.70 muestra un flujo de objeto procedimental en un diagrama de secuencia. Cuando se está modelando un flujo de objeto procedimental, un objeto cede el control al producirse una llamada, hasta el subsiguiente retorno. Las llamadas se muestran mediante una punta de flecha continua y rellena. La punta de una flecha de llamada puede hacer que comience una activación o bien un nuevo objeto. Los retornos se muestran con una línea discontinua. El origen de una flecha de retorno puede hacer que finalice una activación o un objeto.

Las bifurcaciones se muestran partiendo la línea de vida del objeto. Cada bifurcación puede enviar y recibir mensajes. Normalmente, cada rama envía mensajes diferentes. Eventualmente, las líneas de vida del objeto tienen que fusionarse de nuevo.

La Figura 13.71 muestra los estados que aparecen durante la vida de una entrada para algún espectáculo. Las líneas de vida pueden verse interrumpidas por un símbolo de estado para mostrar un cambio de estado. Esto corresponde a una transición de conversión en un diagrama de colaboración. Se puede dibujar una flecha hasta el símbolo de estado para indicar el mensaje que ha dado lugar al cambio de estado.

Gran parte de esta notación se ha extraído directamente de la notación Object Message Sequence Chart (Diagrama de Secuencias de Mensajes de Objetos) propuesta por Buschmann, Meunier, Rohnert, Sommerlad y Stal [Buschmann-96], que a su vez se deriva de la notación Message Sequence Chart (diagrama de secuencias de mensajes).

Un diagrama de secuencia también se puede mostrar en forma de descriptor, en el cual los constituyentes son roles en lugar de objetos. Este diagrama muestra el caso general, no una sola ejecución del mismo. Los diagramas del nivel de descriptores se dibujan sin subrayados, porque los símbolos denotan roles y no objetos individuales.

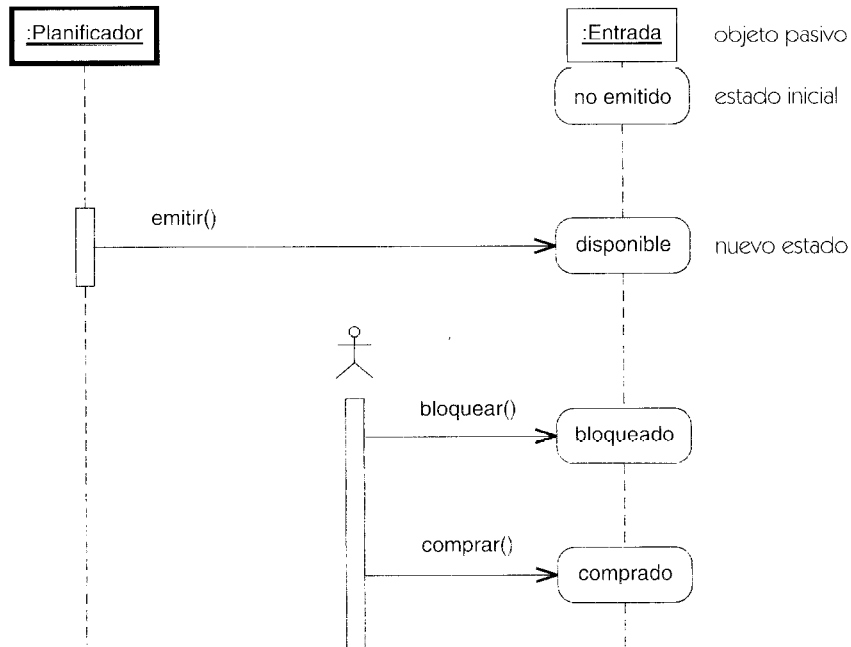


Figura 13.71 Estados de objetos en un diagrama de secuencia

discriminador

Un pseudoatributo que selecciona un elemento hijo de entre un conjunto de hijos en una relación de generalización. Todos los hijos representan una cualidad dada por la que especializar al padre, en contraste a otras cualidades potenciales por las que especializar al mismo padre; representa una dimensión de la especialización.

Véase también generalización, supratipo, pseudoatributo.

Semántica

A veces, un elemento del modelo se puede especializar según diversas cualidades. Cada cualidad representa una dimensión ortogonal independiente de la especialización. Por ejemplo, un vehículo se puede especializar por la propulsión (motor de gasolina, motor espacial, viento, animal, humano), así como por el lugar donde se mueve (tierra, agua, aire, espacio exterior). Un discriminador es el nombre de una dimensión de la especialización. Un elemento se puede especializar en múltiples dimensiones, que deben estar presentes en una instancia concreta.

Una relación de generalización puede tener un discriminador: una cadena que represente una dimensión para clasificar a los hijos del padre. Todas las relaciones de especialización de un solo padre con el mismo nombre de discriminador forman un grupo; cada grupo es una dimensión separada de la especialización. El conjunto completo de nombres del discriminador representa el conjunto completo de dimensiones para especializar al padre. Una instancia debe ser simultáneamente una instancia de un hijo de cada grupo del discriminador. Por ejemplo, un vehículo debe tener una propulsión y un lugar.

Cada discriminador representa una cualidad abstracta del padre, una cualidad que es especializada por los hijos que llevan esa relación del discriminador al padre. Pero un padre con discriminadores múltiples tiene dimensiones múltiples, que se deben especializar para producir un elemento concreto. Por lo tanto, los hijos dentro de un grupo del discriminador son intrínsecamente abstractos. Cada uno de ellos es solamente una descripción parcial del padre, una descripción que acentúe una cualidad e ignore el resto.

Por ejemplo, una subclase de vehículo que se centra en la propulsión omite el lugar. Un elemento concreto requiere la especialización de todas las dimensiones simultáneamente. Esto puede ocurrir por la herencia múltiple del elemento concreto del modelo de un hijo en cada una de las dimensiones, o por la clasificación múltiple de una instancia de un hijo en cada una de las dimensiones. Hasta que se combinan todos los discriminadores, la descripción sigue siendo abstracta.

Por ejemplo, un vehículo real debe tener un medio de propulsión y un lugar. Un vehículo con propulsión por viento y que se mueve por el agua es un barco de vela. No hay nombre particular para un vehículo de tracción animal que se mueva por el aire, pero las instancias de la combinación existen en la fantasía y en la mitología.

La ausencia de una etiqueta de discriminador indica el discriminador «empty» (vacío), que también se considera un discriminador válido (el discriminador por defecto). Esta convención permite que el caso usual no discriminado sea tratado uniformemente. En efecto, si ninguna de las líneas de generalización tiene discriminadores, entonces todos los hijos están en el mismo discriminador. Es decir, hay un discriminador al cual pertenecen todas las especializaciones, y produce la misma semántica que el caso sin discriminador.

Estructura

Cada arco de especialización (generalización) tiene una cadena de discriminador, que puede ser la cadena vacía.

El discriminador es un pseudoatributo del padre. Debe ser único entre los atributos y los roles de la asociación del padre. El dominio para el pseudoatributo es el conjunto de clases hijas. Las múltiples incidencias del mismo nombre de discriminador se permiten entre hijos diferentes y el padre e indican que los hijos pertenecen a la misma partición.

Notación

Un discriminador se muestra como una etiqueta de texto en una flecha de generalización (Figura 13.72). Si dos arcos de generalización con el mismo discriminador comparten una punta de flecha, el discriminador se puede colocar en la punta de flecha.

Ejemplo

La Figura 13.72 muestra una especialización de **Empleado** en dos dimensiones: **estado** del empleado y **localidad**. Cada dimensión tiene el rango de los valores representados por las subclases. Pero son necesarias ambas dimensiones para producir una subclase instanciable. El **Enlace**, por ejemplo, es una clase que es ambos, **Supervisor** y **Expatriado**.

Cualquier descendiente de una sola dimensión es abstracto, hasta que las dos dimensiones se recombinan en un descendiente con herencia múltiple.

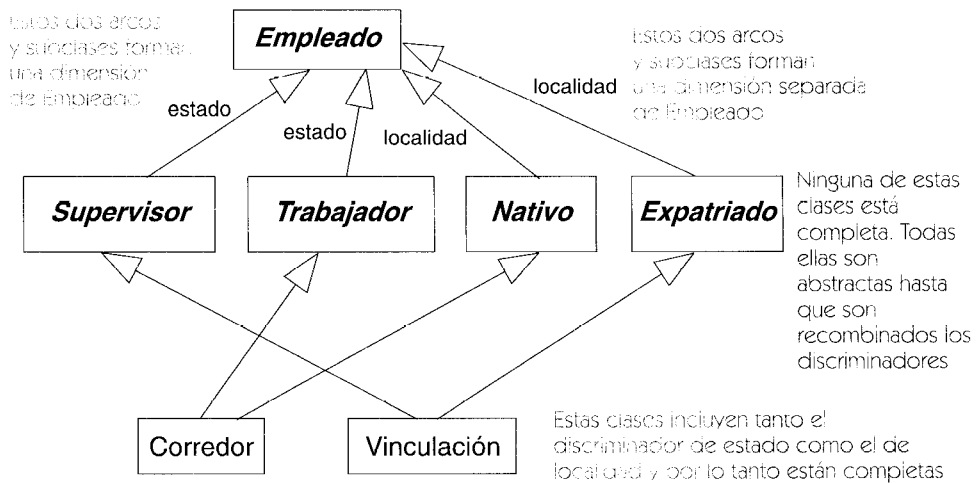


Figura 13.72 Discriminadores

diseño

Esa etapa de un sistema que describe cómo se implementará el sistema, en un nivel lógico sobre código real. En el diseño, las decisiones estratégicas y tácticas se toman para resolver los requisitos funcionales y de calidad requeridos de un sistema. Los resultados de esta etapa son representados por los modelos a nivel de diseño, especialmente la vista estática, vista de la máquina de estados, y vista de interacción. Contraste con: análisis, diseño, implementación, y despliegue.

Véase fases de modelado, proceso de desarrollo.

disparador

Evento cuya aparición hace que una transición pueda dispararse. Se puede emplear la aplicación como nombre (para denotar el efecto en sí) o como verbo (para denotar la aparición del evento³).

Véase también transición de finalización, transición.

Semántica

Toda transición (salvo una transición de finalización, que se dispara al finalizar alguna actividad interna) hace referencia a un evento que forma parte de su estructura. Si se produce el evento

³ En inglés, la palabra "trigger" significa tanto "disparador" como "disparar". N. del T.

cuando el objeto se encuentra en un estado que contenga una transición saliente cuyo disparador sea ese evento o un antecesor del evento, entonces se comprueba la condición de guarda de la transición. Si se cumple la condición, entonces se permite que se dispare la transición. Si hay más de una transición disponible para ser disparada, sólo llega a dispararse una. La selección puede no ser determinista. (Si el objeto posee más de un estado concurrente, se puede disparar una transición desde cada estado, pero como máximo se puede disparar una transición desde cada estado.)

Observe que la condición de guarda sólo se comprueba una vez, en el momento en que se produce el evento disparador (incluyendo los eventos implícitos de finalización). Si no se habilita para el disparo ninguna transición al producirse un evento, entonces se ignora el evento. Esto no es un error.

Los parámetros del evento disparador se pueden utilizar en la condición de guarda o en una acción asociada a la transición, así como en una acción de entrada en el estado destino.

A lo largo de la ejecución de un paso que se ejecuta hasta finalizar realizado después de una transición, el evento disparador sigue estando disponible como evento actual para las acciones de los subpasos de la transición. El tipo exacto de este evento puede ser desconocido en una acción de entrada o en algún segmento posterior de una transición de múltiples segmentos. Por tanto, el tipo de evento se puede discriminar en la acción empleando una operación polimorfa o una sentencia de selección (como *case* o *switch*). Una vez conocido el tipo exacto del evento, se pueden utilizar sus parámetros.

Notación

El nombre y signatura del evento disparador forman parte de la etiqueta de la transición.

Véase transición.

Se puede acceder al evento disparador desde una expresión mediante la palabra clave **currentEvent**. Esta palabra clave se refiere al evento disparador del primer segmento en las transiciones de múltiples segmentos.

disparar

Ejecutar una transición.

Véase también atómico/a, disparador.

Semántica

Cuando ocurre un evento requerido por una transición, y la condición de guarda en la transición está satisfecha, la transición realiza su acción y cambia el estado activo.

Cuando un objeto recibe un evento, se guarda el evento si la máquina de estados está ejecutando una etapa atómica. Cuando se termina la etapa, la máquina de estados gestiona el evento que ha ocurrido. Una transición se activa si su evento se maneja mientras el propio

objeto está en el estado que contiene la transición o en un subestado anidado dentro del estado que contiene la transición. Un evento satisface un evento de disparador cuyo tipo sea un antecesor del tipo del evento que ocurre. Si una transición compleja tiene múltiples estados de origen, todos deben estar activos para que la transición esté disponible. Una transición de terminación se activa cuando su estado origen termina su actividad. Si es un estado compuesto, está disponible cuando todos sus subestados directos han terminado o han alcanzado sus estados finales.

Cuando se trata el evento, se evalúa la condición de guarda (si existe). Si la expresión booleana en la condición de guarda se evalúa como verdadera, entonces la transición se dispara. La acción en la transición se ejecuta, y el estado del objeto se convierte en el estado destino de la transición (sin embargo, no ocurre ningún cambio de estado en una transición interna). Durante el cambio de estado, cualquiera que sean acciones de salida y acciones de entrada en la ruta mínima, desde el estado original del objeto al estado destino, se ejecutan las transiciones. Observe que el estado original puede ser un subestado anidado del estado origen de la transición.

Si la condición de guarda no está satisfecha, nada sucede como resultado de esta transición, aunque alguna otra transición podría dispararse si sus condiciones están satisfechas.

Aunque más de una transición sea elegible para ser disparada, sólo una de ellas se disparará. Una transición en un estado anidado tiene prioridad sobre una transición en un estado que incluye. Si no, la elección de transición es indefinida y puede ser no determinista. Esto es a menudo una situación del mundo real.

Como cuestión práctica, una implementación puede proporcionar una ordenación de las transiciones para dispararlas. Esto no cambia la semántica, pues el mismo efecto podría ser alcanzado organizando las condiciones de guarda de modo que no se solapen. Pero es a menudo más simple poder decir "Esta transición solamente se dispara si ninguna otra transición se dispara".

Eventos diferidos. Si el evento o uno de sus antecesores está marcado como diferido en el estado o en un estado lo que incluye, y el evento no acciona una transición, el evento es un evento diferido hasta que el objeto pasa a un estado en el cual el evento no se difiera. Cuando el objeto pasa a un nuevo estado, cualquier evento previamente diferido pendiente que ya no sea diferido, pasa a estar pendiente y ocurren en un orden indeterminado. Si el primer evento pendiente no causa que se dispare una transición, se ignora y otro ocurre evento pendiente. Si un evento previamente diferido está marcado para el aplazamiento en el nuevo estado, puede accionar una transición, pero sigue diferido si no puede accionar una transición. Si la ocurrencia de un evento causa una transición a un nuevo estado, cualquier evento pendiente restante o evento diferido son evaluados de acuerdo con el estado de aplazamientos del nuevo estado y se establece un nuevo conjunto de eventos pendientes.

Una implementación puede imponer reglas más terminantes sobre el orden en el que los eventos diferidos se procesan u ofrecer operaciones para manipular dicho orden.

división

Una transición compleja en la cual un estado origen es sustituido por dos o más estados destino, dando por resultado un aumento en el número de estados activos. Antónimo: unión.

Véase también transición compleja, estado compuesto, unión.

Semántica

Una división es una transición con un estado origen y dos o más estados de destino. Si el estado origen está activo y ocurre el evento disparador, se ejecuta la acción de la transición y todos los estados destino pasan a estar activos. Los estados destino deben estar en diversas regiones de un estado compuesto concurrente.

Notación

Una división se representa como una línea gruesa a la que llega una flecha de la transición y salen dos o más flechas de transición. Puede tener una etiqueta de transición (condición de guarda, evento disparador, y acción). La Figura 13.73 muestra una división explícita a un estado compuesto concurrente.

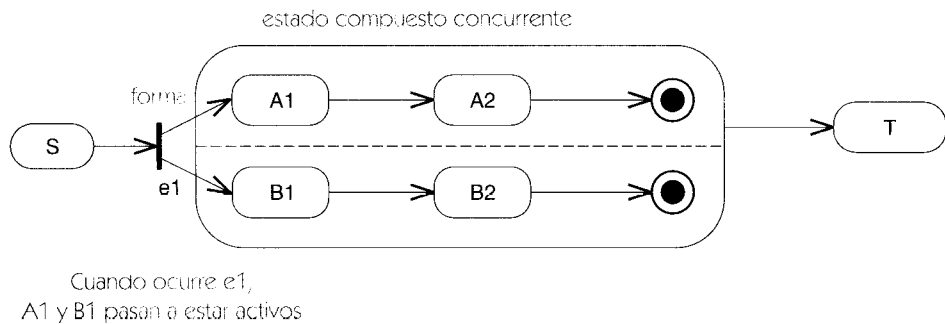


Figura 13.73 División

eficiencia de navegación

Término que indica si es o no posible recorrer eficientemente una asociación binaria partiendo de un objeto para obtener el objeto o conjunto de objetos asociados con él. El concepto no es aplicable a las asociaciones n -arias. La eficiencia de navegación está relacionada con la navegabilidad, pero no es una propiedad definitoria.

Véase también navegabilidad.

Semántica

Se puede definir la eficiencia de navegación en forma general, para que sea aplicable al diseño abstracto y también a distintos lenguajes de programación. Diremos que una asociación binaria es navegable eficientemente si el coste medio necesario para obtener el conjunto de objetos asociados es proporcional al número de objetos del conjunto (no al límite superior de la multiplicidad, que podría ser infinito) más una constante determinada. En términos de complejidad computacional, el coste es $O(n)$. Si la multiplicidad es uno o cero-uno, entonces el coste de acceso tiene que ser constante, lo cual impide efectuar búsquedas en una lista de longitud variable. Una definición ligeramente más imprecisa de la eficiencia de navegación permitiría un coste mínimo de $\log(n)$.

Aun cuando las asociaciones navegables de multiplicidad uno suelen implementarse empleando un puntero incrustado en el bloque que contiene los atributos del objeto, es posible una implementación externa empleando tablas hash, que tienen un coste medio de acceso constante. Por tanto, se puede implementar una asociación como un objeto de tabla de búsqueda externo con respecto a las clases participantes y se puede seguir considerando navegable. (En algunas situaciones de tiempo real, es preciso limitar el tiempo en el caso peor y no el tiempo medio. Esto no requiere un cambio en la definición básica salvo cambiar el tiempo medio por el tiempo en el caso peor, pero pueden ser descartados los algoritmos probabilísticos como las tablas hash.)

Si una asociación no es navegable en una dirección dada, esto no significa que no se pueda recorrer en absoluto, sino que el coste del recorrido puede ser significativo —por ejemplo, quizá requiera efectuar búsquedas en una larga lista—. Si el acceso en una dirección es infrecuente, una búsqueda puede ser una opción razonable. La eficiencia de navegación es un concepto de diseño que permite al diseñador diseñar las rutas de acceso a objetos teniendo una comprensión de los costes de complejidad computacional. Normalmente, la navegabilidad implica la eficiencia de navegación.

Resulta posible (aun siendo relativamente raro) tener una asociación que no sea navegable eficientemente en ninguna dirección. Esta asociación podría estar implementada como una lista de enlaces que es preciso estudiar para realizar un recorrido en cualquier dirección. Sería posible recorrerla, pero sería poco eficiente. Sin embargo, la utilidad de una de estas asociaciones es reducida.

Discusión

La eficiencia de navegación indica la eficiencia que se tiene al obtener el conjunto de objetos que están relacionados con un objeto dado. Cuando la multiplicidad es 0..1 ó 1, entonces la implementación evidente es un puntero en el objeto origen. Cuando la multiplicidad es muchos, entonces la implementación normal es una clase contenedora que alberga un conjunto de punteros. La clase contenedora, a su vez, puede residir o no en el registro de datos de un objeto de la clase, dependiendo de si se puede o no obtener con coste constante (que es lo que suele suceder para el acceso a través de punteros). La clase contenedora debe tener la propiedad de eficiencia de navegación. Por ejemplo, una lista simple de todos los enlaces de una asociación no sería eficiente, porque los enlaces de un objeto estarían mezclados con muchos otros enlaces carentes de interés y sería preciso hacer una búsqueda. Una lista de enlaces almacenada en cada objeto sí sería eficiente, porque no se requeriría una búsqueda innecesaria.

En una asociación cualificada, una indicación de navegabilidad en la dirección que sale del calificador suele indicar que es eficiente obtener el objeto o conjunto de objetos seleccionado por un objeto origen y un valor del calificador. Esto es consistente con una implementación que haga uso de tablas hash o quizá con un árbol binario indexado por el valor del calificador (que es precisamente la razón por la que se incluyen los calificadores como concepto de modelado).

ejecutar hasta finalizar

Transición o serie de acciones que tiene que acabar por completo.

Véase también acción, atómico/a, máquina de estados, transición.

Semántica

En una máquina de estados, ciertas acciones o series de acciones son atómicas: no se pueden hacer acabar, ni abortar, ni pueden ser interrumpidas por otras acciones. Cuando se dispara una transición, todas las acciones asociadas a ella o que hayan sido invocadas por ella deben terminarse como un grupo, incluyendo las acciones de entrada y las acciones de salida de aquellos estados en los que entre o de los que salga. Se dice que la ejecución de una transición se ejecuta hasta finalizar porque no espera para admitir otros eventos.

La semántica de ejecutar-hasta-finalizar puede contrastarse con la semántica de los estados normales. Cuando está activo un estado, un evento puede producir una transición a otro estado. Toda actividad de ese estado será abortada por la transición.

Una transición puede estar compuesta por múltiples segmentos organizados en forma de cadena y separados mediante pseudoestados. Un grupo de varias cadenas se puede separar o fusionar, así que el modelo global puede contener un grafo de segmentos separados por pseudoestados. Sólo el primer segmento de la cadena puede tener un evento disparador. La transición se dispara cuando la máquina de estados maneja el evento disparado. Si se satisfacen las condiciones de guarda de todos los segmentos, entonces la transición está habilitada y se dispara, siempre y cuando no se dispare otra transición. Se ejecutan las acciones de los sucesivos segmentos. Una vez comenzada la ejecución, es preciso ejecutar las acciones de todos los segmentos antes de que haya finalizado el paso de ejecución hasta finalizar.

Durante la ejecución de un paso que se ejecuta hasta finalizar, el evento disparador que haya iniciado la transición está disponible para las acciones como evento en curso. Las acciones de entrada y salida pueden por tanto obtener argumentos del evento disparador. Los distintos eventos pueden dar lugar a una acción de entrada o de salida, pero una acción puede discriminar el tipo de suceso en curso en una sentencia de caso.

Como consecuencia de la semántica que poseen las acciones en el caso de una ejecución hasta finalización, debería utilizarse para modelar asignaciones, indicadores, aritmética sencilla y otros tipos de operaciones de mantenimiento doméstico. Los cálculos largos deberían modelarse como actividades interrumpibles.

elaboración

La segunda fase de un proceso de desarrollo de software, durante el cual el diseño del sistema ha empezado y la arquitectura es desarrollada y probada. Durante esta fase, se completa la mayor parte de la vista del análisis, junto con las partes arquitectónicas de la vista del diseño. Si se construye un prototipo ejecutable, se hace algo de la vista de la implementación.

Véase proceso de desarrollo.

elemento

Un componente atómico de un modelo. Este libro describe los elementos que pueden utilizarse en los modelos de UML —elementos del modelo, que expresan información semántica, y

elementos de presentación, que proporcionan representaciones gráficas de la información del elemento modelo.

Véase también diagrama, elemento modelo.

Semántica

El término *elemento* es tan amplio que tiene poca semántica específica.

Todos los elementos pueden tener la siguiente propiedad.

valor etiquetado	Se pueden unir cero o más pares etiqueta-valor a cualquier elemento. La etiqueta es un nombre que identifica el significado del valor. Las etiquetas no son fijas en UML sino que se pueden extender para denotar varios tipos de información significativa para el modelador o para una herramienta de edición.
------------------	--

Elementos estándar

documentation.

elemento modelo

Dícese de un elemento que es una abstracción extraída del sistema que se está modelando. Compárese con elemento de presentación, que es una presentación (generalmente visual) de uno o más elementos de modelado para la interacción humana.

Semántica

Todos los elementos que tienen semántica son elementos del modelo, incluyendo los conceptos del mundo real y los conceptos de implementación de un sistema de computadores. Los elementos gráficos que tienen como propósito visualizar un modelo son elementos de representación. No son elementos del modelo, por cuanto no añaden semántica al mismo.

Los elementos modelo pueden poseer nombres, pero el uso y restricciones relativos a los nombres varían según la clase de elemento modelo y se discuten para cada una de estas clases. Todo elemento modelo pertenece a un espacio de nombres adecuado para la clase del elemento. Todos los elementos modelo pueden tener asociadas las propiedades siguientes:

valor etiquetado	Es posible asociar cero o más parejas valor-etiqueta a cualquier elemento modelo o de representación. La etiqueta es un nombre que identifica el significado del valor. Las etiquetas no están fijadas en UML pero se pueden extender para denotar distintas clases de información significativas para quien crea el modelo o para alguna herramienta de edición.
------------------	---

restricción	Es posible asociar cero o más restricciones a los elementos modelo. Las restricciones son limitaciones que se expresan como cadenas lingüísticas en un lenguaje de restricción.
estereotipo	Se pueden asociar cero o más estereotipos a un elemento modelo, siempre y cuando el estereotipo sea aplicable al elemento base del modelo. El estereotipo no altera la estructura de la clase base, pero puede agregar restricciones y valores etiquetados que sean aplicables a los elementos del modelo que lleven ese estereotipo.

Además, los elementos del modelo pueden participar en relaciones de dependencia.

Véase el Capítulo 14, Elementos Estándar, para estudiar una lista de etiquetas, restricciones y estereotipos predefinidos.

elemento de representación

Dícese de la proyección textual o gráfica de uno o más elementos del modelo. *Véase también* diagrama.

Semántica

Los elementos de representación (que a veces se llaman elementos de visualización, aunque incluyen formas de representación que no son gráficas) presentan la información de un menú desplegable de forma adecuada para su percepción por seres humanos. Son la notación. Muestran toda la información semántica relativa a un elemento del modelo, o parte de ella. También pueden agregar información estética útil para las personas, por ejemplo agrupando aquellos elementos que están relacionados conceptualmente. Pero la información añadida no posee contenido semántico. Lo que se espera es que los elementos de representación sean responsables de mantenerse correctamente a sí mismos a pesar de los cambios experimentados por el modelo subyacente; sin embargo, los elementos del modelo no tienen por qué ser conscientes de los elementos de presentación para funcionar correctamente.

Las descripciones de la notación UML de este libro definen la correspondencia entre elementos del modelo y representaciones gráficas en una pantalla. La implementación de los elementos de representación como objetos es responsabilidad de la implementación de las herramientas.

elemento derivado

Un elemento que se puede calcular a partir de otros elementos y es incluido por claridad o por propósitos del diseño aunque no añade ninguna información semántica.

Véase también restricción, dependencia.

Semántica

Un elemento derivado es lógicamente redundante dentro de un modelo porque puede ser calculado a partir de uno o más elementos diferentes. La fórmula para calcular un elemento derivado se puede dar como restricción.

Un elemento derivado puede ser incluido en un modelo por varias razones. En el análisis, un elemento derivado es semánticamente innecesario, pero puede ser utilizado para proporcionar un nombre o una definición para un concepto significativo, como un tipo de macro. Es importante recordar que un elemento derivado no agrega nada a la semántica de un modelo.

En un modelo de diseño, un elemento derivado representa una optimización —un elemento que puede ser calculado a partir de otros elementos pero que está físicamente presente en el modelo para evitar el coste o el problema del recálculo—. Los ejemplos son un valor intermedio de un cálculo o un índice a un conjunto de valores. La presencia de un elemento derivado implica la responsabilidad de actualizarlo si los valores de los que depende cambian.

Notación

Un elemento derivado se representa mediante una barra inclinada (/) delante del nombre del elemento derivado, tal como una atributo, un nombre de rol, o de un nombre de la asociación (Figura 13.74).

Los detalles sobre cómo calcular un elemento derivado se pueden especificar por una dependencia con el estereotipo «**derive**». Generalmente, es conveniente suprimir en la notación la flecha de la dependencia que va desde la restricción al elemento y colocar simplemente una cadena de restricción cerca del elemento derivado, aunque la flecha se puede incluir cuando sea provechoso.

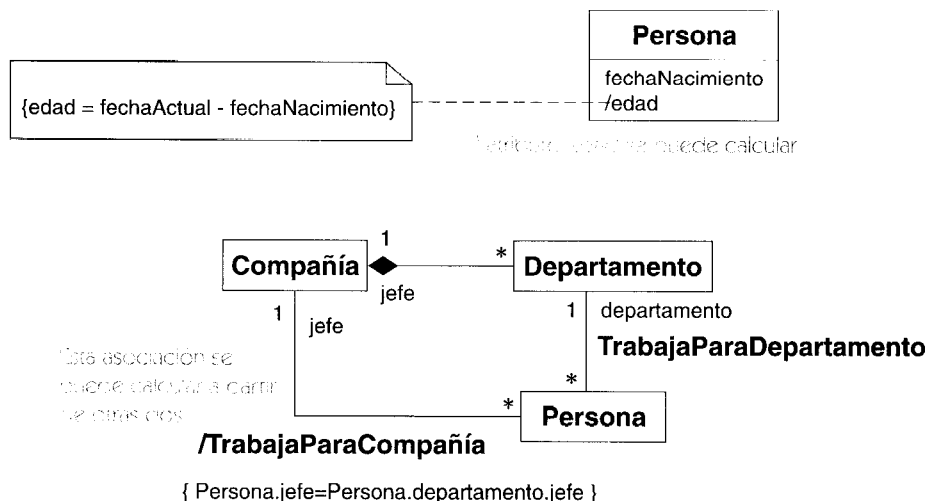


Figura 13.74 Atributo derivado y asociación derivada

Discusión

Las asociaciones derivadas son probablemente el tipo más común de elemento derivado. Representan asociaciones virtuales que se pueden calcular a partir asociaciones de dos o más asociaciones fundamentales. En la Figura 13.74, por ejemplo, la asociación derivada **TrabajaParaCompañía** puede ser calculada componiendo **TrabajaParaDepartamento** con la composición **jefe**.

Una implementación puede incluir explícitamente **TrabajaParaCompañía** para evitar el recálculo, pero no representa ninguna información adicional.

Hay una diferencia con la generalización de la asociación (Figura 13.112), que representa dos niveles de detalle para una asociación. Normalmente no serán implementados ambos niveles. Generalmente solamente las asociaciones del hijo serán implementadas. A veces solamente se implementará la asociación del padre, con las asociaciones del hijo limitando los tipos de objetos que pueden relacionarse.

elemento generalizable

Un elemento del modelo que puede participar en una relación de generalización.

Véase también generalización, herencia.

Semántica

Un elemento generalizable puede tener padres e hijos. Una variable que se clasifica con un elemento puede sostener una instancia de un descendiente del elemento.

Los elementos generalizables incluyen clases, casos de uso, otros clasificadores, asociaciones, estados, eventos, y colaboraciones. Un elemento generalizable hereda las características de sus antecesores. La definición de qué partes de cada tipo de elemento generalizable se heredan, depende del tipo del elemento. Las clases, por ejemplo, heredan atributos, operaciones, métodos, participación en asociaciones, y restricciones. Las asociaciones heredan las clases participantes (éstas pueden especializarse por sí mismas) y las propiedades del extremo de la asociación. Los casos de uso heredan los atributos y operaciones, asociaciones a actores, relaciones de inclusión y extensión a otros casos de uso, y secuencias de comportamiento. Los estados heredan transiciones.

Véase generalización, generalización de asociaciones, generalización de casos de uso.

Estructura

Un elemento generalizable tiene propiedades que declaran dónde puede aparecer dentro de una jerarquía de generalización.

abstraction	Especifica si el elemento generalizable describe instancias directas o es un elemento abstracto que debe ser especializado antes de que pueda ser instanciado. True indica que el elemento es abstracto (no puede tener instancias directas); false indica que es concreto (puede tener instancias
-------------	--

directas). Para ser utilizable, un elemento abstracto debe tener descendientes concretos. Una clase con una operación que carece de métodos es abstracta por necesidad.

leaf	Especifica si el elemento generalizable puede ser especializado. True indica que no puede tener descendientes (es decir, debe ser una hoja); false indica que puede tener descendientes (tanto si los tiene actualmente como si no). Una clase abstracta que es una hoja, es inútil para todo menos para agrupar atributos y operaciones globales.
root	Especifica si el elemento debe ser una raíz sin antecesores. True indica que debe ser una raíz y no puede tener antecesores; false indica que no necesita ser una raíz y puede tener antecesores (tanto si tiene antecesores en ese momento como si no).

Obsérvese que el declarar hojas y clases raíz no afecta a la semántica, pero tales declaraciones pueden proporcionar una declaración de las intenciones del diseñador. Pueden también permitir una compilación más eficiente de paquetes separados evitando la necesidad de un análisis global o de asunciones excedentemente conservadoras sobre operaciones polimórficas.

Elementos estándar

leaf.

elemento ligado

Elemento del modelo que se crea enlazando los parámetros de una plantilla con unos valores de los argumentos.

Véase también ligadura, plantilla.

Semántica

Una plantilla es una descripción parametrizada de un grupo potencial de elementos. Para obtener un elemento real, hay que *ligar* los parámetros de la plantilla con los valores reales. El valor real de cada parámetro es una expresión proporcionada por el alcance en el que sucede la ligadura. La mayoría de los argumentos son o clases o enteros.

Si el alcance es en sí mismo una plantilla, los parámetros de la plantilla externa pueden ser utilizados como argumentos para ligar la plantilla original, volviendo a parametrizarla. Pero los nombres de los parámetros de una plantilla no tienen sentido fuera del cuerpo de la misma. No se puede suponer que los parámetros de dos plantillas se corresponden sólo porque tienen los mismos nombres, al igual que no se asume que los parámetros de las subrutinas se corresponden con sus argumentos sólo basándose en los nombres.

Un elemento ligado está completamente especificado por su plantilla, por lo que su contenido no puede ampliarse (extenderse). No se permite la declaración de nuevos atributos u operaciones para las clases, pero por ejemplo, de una clase ligada podrían derivarse subclases y las subclases extenderse de la manera habitual.

Ejemplo

La Figura 13.75 muestra cómo ligar más de una vez una plantilla. **ArrayPuntos** es una plantilla con un parámetro, el tamaño **n**. Queremos construirla a partir de una plantilla existente, **FArray**, que tiene dos parámetros, el tipo de los elementos **T** y el tamaño **k**. Para hacerlo, el parámetro **k** de la plantilla **FArray** se liga con el parámetro **n** de la plantilla **ArrayPuntos**, y el parámetro **T** de la plantilla **FArray** se liga con la clase **Punto**. Al hacer esto, se produce el efecto de borrado de un parámetro de la plantilla original. Si ahora queremos utilizar la plantilla **ArrayPuntos** para construir una clase **Triángulo**, el parámetro que indica el tamaño **n**, se liga con el valor 3. Para construir una clase **Cuadrilátero**, se liga con el valor 4, y así sucesivamente.

La Figura 13.75 también muestra una plantilla **Polígono**, hija de la clase **Figura**. Esto significa que cada clase construida a partir de la plantilla **FArray** es una subclase de **Figura**: las clases **Triángulo** y **Cuadrilátero** son ambas subclases de **Figura**.

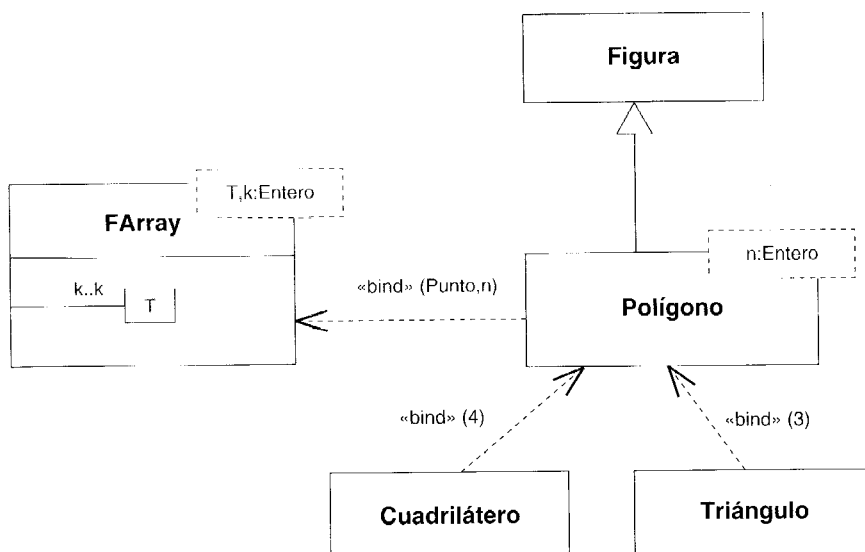


Figura 13.75 Ligado doble de una plantilla

Notación

Un elemento ligado se representa utilizando una flecha discontinua con la palabra clave «bind» que parte del elemento ligado y que apunta a la plantilla. Una forma de representación alternativa es utilizar la sintaxis textual `NombrePlantilla<argumento...>`, empleando la correspondencia de nombres para identificar la plantilla. La forma textual evita tener que mostrar la plantilla y dibujar la flecha, por lo que es particularmente útil cuando el elemento ligado se utiliza como clasificador para un atributo o parámetro de una operación.

Véase ligadura para más detalles. La Figura 13.130 muestra un ejemplo.

En una clase ligada, los apartados de atributos y operaciones normalmente se suprimen ya que no pueden modificarse en los elementos ligados.

El nombre de un elemento ligado, bien en la forma “anónima” entre símbolos menor y mayor que, o bien en la forma explícita de “ligadura con flecha”, se puede utilizar en cualquier parte donde se pueda utilizar el nombre de un elemento del tipo parametrizado. Por ejemplo, un nombre de clase ligada podría utilizarse como tipo de un atributo o en la signatura de una operación dentro de un símbolo de clase en un diagrama. La Figura 13.76 muestra un ejemplo de todo esto.

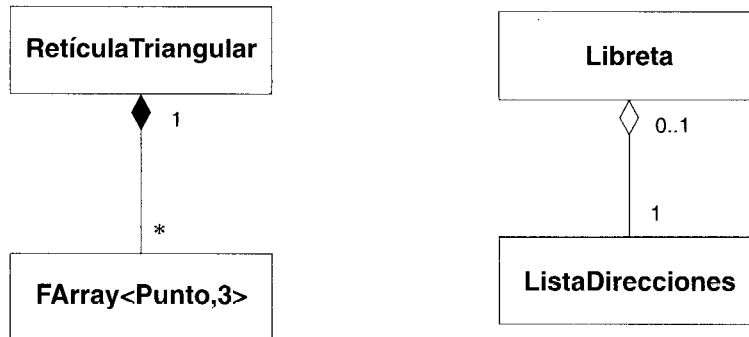


Figura 13.76 Uso de plantillas ligadas en asociaciones

Discusión

Los clasificadores son candidatos obvios a la parametrización. Los tipos de sus atributos, operaciones o clasificadores asociados son parámetros muy comunes en las plantillas. Las colaboraciones parametrizadas se denominan patrones. Las operaciones, en cierto sentido, son inherentemente parametrizadas. La utilidad de la parametrización de otros elementos no está tan clara, pero posiblemente se le encontrarán otros usos en el futuro.

elemento parametrizado

Véase plantilla.

emisor

Objeto que pasa una instancia de mensaje a un objeto receptor.

Véase llamada, enviar.

enlace

Una tupla de referencias de objetos que es una instancia de una asociación o un rol de asociación.

Semántica

Un enlace es una conexión individual entre dos o más objetos. Se trata de una tupla (lista ordenada) de referencias de objetos. Los objetos deben ser instancias directas o indirectas de las clases situadas en las posiciones correspondientes de la asociación. Una asociación no puede contener enlaces duplicados procedentes de la misma asociación —esto es, dos tuplas idénticas de referencias de objetos.

Un enlace que sea una instancia de una clase de asociación puede tener una lista de valores de atributos además de la tupla de referencias de objetos. No se permiten los enlaces duplicados con la misma tupla de referencias de objetos, aun cuando los valores de sus atributos sean distintos. La identidad de un enlace proviene de su tupla de referencias de objetos, que debe ser única.

Se puede emplear un lenguaje para la navegación. En otras palabras, un objeto que aparece en una posición en un enlace puede obtener el conjunto de objetos que aparecen en otra posición. Entonces puede enviarles mensajes (esto se denomina “enviar un mensaje a través de una asociación”). El proceso es eficiente si la asociación tiene la propiedad de navegabilidad en la dirección del destino. El acceso puede o no ser posible si la asociación no es navegable, pero es probable que sea ineficiente. La navegabilidad en direcciones opuestas se especifica de manera independiente.

Dentro de una colaboración, un rol de asociación es una relación contextual, frecuentemente transitoria, entre clasificadores. Una instancia de rol de asociación también es un enlace, pero típicamente se trata de uno cuya vida está limitada a la duración de la colaboración.

Notación

Los enlaces binarios se representan mediante una ruta entre dos objetos —esto es, uno o más segmento de línea o arcos—. En el caso de una asociación reflexiva, la ruta es un bucle cuyos dos extremos llegan a un único objeto.

Véase asociación para más detalles acerca de las rutas.

Puede mostrarse un nombre de rol en el extremo de cada enlace. Se puede mostrar un nombre de asociación junto a la ruta. Si está presente, se subraya el nombre para indicar que es una instancia. Los enlaces poseen nombres de instancia. Toman su identidad de los objetos que relacionan. No se muestra multiplicidad para los enlaces porque las instancias *no* poseen multiplicidad; la multiplicidad es una propiedad del descriptor que limita el número de instancias que pueden existir. Es posible mostrar otros adornos de asociación (agregación, composición y navegación) en los roles de enlace.

Se puede mostrar un calificador en un enlace. El valor de calificador se puede mostrar en su cuadro. La Figura 13.77 muestra tanto enlaces ordinarios como enlaces cualificados.

Otros adornos de los enlaces pueden mostrar propiedades de sus asociaciones, incluyendo el sentido de navegación, la agregación o composición, los estereotipos de implementación y la visibilidad.

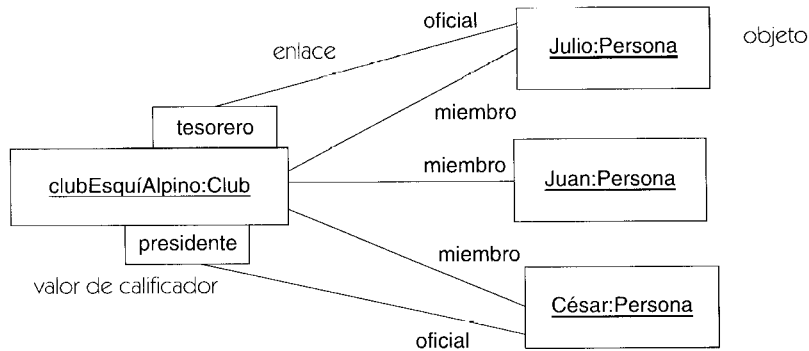


Figura 13.77 Enlaces

Enlace *n*-ario. Un enlace *n*-ario se representa mediante un rombo con una ruta que va hacia cada uno de los objetos participantes. Los demás adornos de la asociación y los adornos de los roles tienen las mismas posibilidades que un enlace binario.

Discusión

¿Cómo debería mostrarse una dependencia en un diagrama de objetos? En general, una dependencia representa una relación entre clases, no entre objetos, y pertenece a un diagrama de clases, no a un diagrama de objetos. ¿Qué sucede con los argumentos de procedimiento, las variables locales de los procedimientos y quien haga la llamada a una operación? Todos ellos deberían existir como estructuras de datos reales y no sólo como simples dependencias. Por tanto, se pueden mostrar como enlaces. El que haga una llamada a un procedimiento necesitará una referencia del objeto destino —esto es, un enlace—. Algunos enlaces pueden ser instancias de roles de asociación en las colaboraciones, tal como la mayoría de los parámetros y variables locales. Las dependencias restantes son relevantes para la clase en sí y no para sus objetos individuales.

enlace transitorio

Enlace que existe durante un período limitado, tal como la ejecución de una operación.

Véase también asociación, colaboración, uso.

Semántica

Durante la ejecución, algunos enlaces existen durante un período limitado. Por supuesto, casi todos los objetos o enlaces poseen una duración limitada, si el período de tiempo considerado es suficientemente grande. Algunos enlaces, sin embargo, existen únicamente en ciertos contextos limitados, tales como la ejecución de un método. Los argumentos de procedimientos y las variables locales se pueden representar como enlaces transitorios. Se pueden modelar

todos estos enlaces como asociaciones, pero entonces hay que enunciar de forma muy imprecisa las condiciones de las asociaciones y pierden gran parte de su precisión para restringir las combinaciones de objetos. En lugar de hacer esto, esas situaciones se pueden modelar empleando colaboraciones, que son configuraciones de objetos y enlaces que existen en contextos especiales.

Se puede considerar que un rol de asociación de una colaboración es un enlace transitorio que existe únicamente durante la ejecución de una entidad de comportamiento, tal como un procedimiento. Aparece dentro del modelo de clases como una dependencia de uso. Para disponer de todos los detalles es preciso consultar el modelo de comportamiento.

Notación

Los enlaces transitorios se representan mediante una asociación con un estereotipo junto al rol de enlace para indicar distintos tipos de implementación. Se pueden utilizar los estereotipos siguientes:

« parameter »	Parámetro de procedimiento.
« local »	Variable local de un procedimiento.
« global »	Variable global (algo visible dentro de todo un modelo o paquete); evítese, si es posible, por violar el espíritu de la orientación a objetos.
« self »	Autoenlace (capacidad de un objeto para enviarse mensajes a sí mismo, implícita en los objetos y sólo resulta útil mostrarlo en situaciones dinámicas con flujos de mensajes).
« association »	Asociación (por defecto, es innecesario especificarlo salvo para hacer hincapié); no es un enlace transitorio pero se enumera por compleción.

enumeración

Un tipo de dato, cuyas instancias forman una lista de valores literales con nombre. Generalmente, se declaran el nombre de la enumeración y sus valores literales.

Véase también clasificador, tipo de dato.

Semántica

Una enumeración es un tipo de dato definible por el usuario. Tiene un nombre y una lista ordenada de los nombres literales de la enumeración, cada uno de los cuales es un valor en el rango del tipo de dato —es decir, es una instancia predefinida del tipo de dato—. Por ejemplo, **RGBColor** = {**rojo**, **verde**, **azul**}. El tipo de dato **Booleano** es una enumeración predefinida con los literales **false** y **true**.

Notación

Una enumeración se representa mediante un rectángulo con la palabra clave «**enum**» sobre el nombre de la enumeración en el compartimento superior (Figura 13.78). El segundo compar-

«enum» Booleano	nombre de la enumeración
falso verdadero	literales de enumeración, en orden
and(con:Booleano):Booleano or(con:Booleano):Booleano not():Booleano	operaciones sobre la enumeración (todas consultas) opción unaria, devuelve otro valor enumerado

Figura 13.78 Declaración de una enumeración

timento contiene una lista de nombres literales de la enumeración. El tercer compartimento (si existe) contiene un conjunto de operaciones sobre el tipo. Todas deben ser consultas (las cuales, por lo tanto, no necesitan ser declaradas explícitamente como tales).

enviar

Crear una instancia de señal en un objeto emisor y transferirla a un objeto receptor para aportar información.

Una dependencia de uso de envío relaciona una operación o método que envía una señal o a una clase que contiene esa operación o método con la clase que recibe la señal.

Véase también señal.

Semántica

Un envío es una operación especial que puede realizar un objeto. Especifica la señal que se envía, una lista de argumentos de la señal y un conjunto de objetos destino que deben recibir la señal.

Los objetos envían señales a un conjunto de objetos; con frecuencia, se trata de un conjunto que contiene un único objeto. Se puede considerar una “emisión” como el envío de un mensaje a todos los objetos, aunque en la práctica podría ser implementada como un caso especial, para mayor eficiencia. Si el conjunto de objetos destino contiene más de un objeto, se envía una copia de la señal a todos los objetos del conjunto concurrentemente. Si el conjunto está vacío, no se envía la señal. Esto no es un error.

La creación de un nuevo objeto se puede considerar como enviar un mensaje a un objeto factoría, tal como una clase, que crea la nueva instancia y después le pasa el mensaje como “evento de nacimiento”. Esto proporciona un mecanismo para que el creador se comunique con su creación; el evento de nacimiento se puede considerar como algo que va del creador al nuevo objeto, con el efecto secundario consistente en instanciar el nuevo objeto de pasada. La Figura 13.79 muestra la creación de un objeto empleando tanto la sintaxis de texto como su versión gráfica.

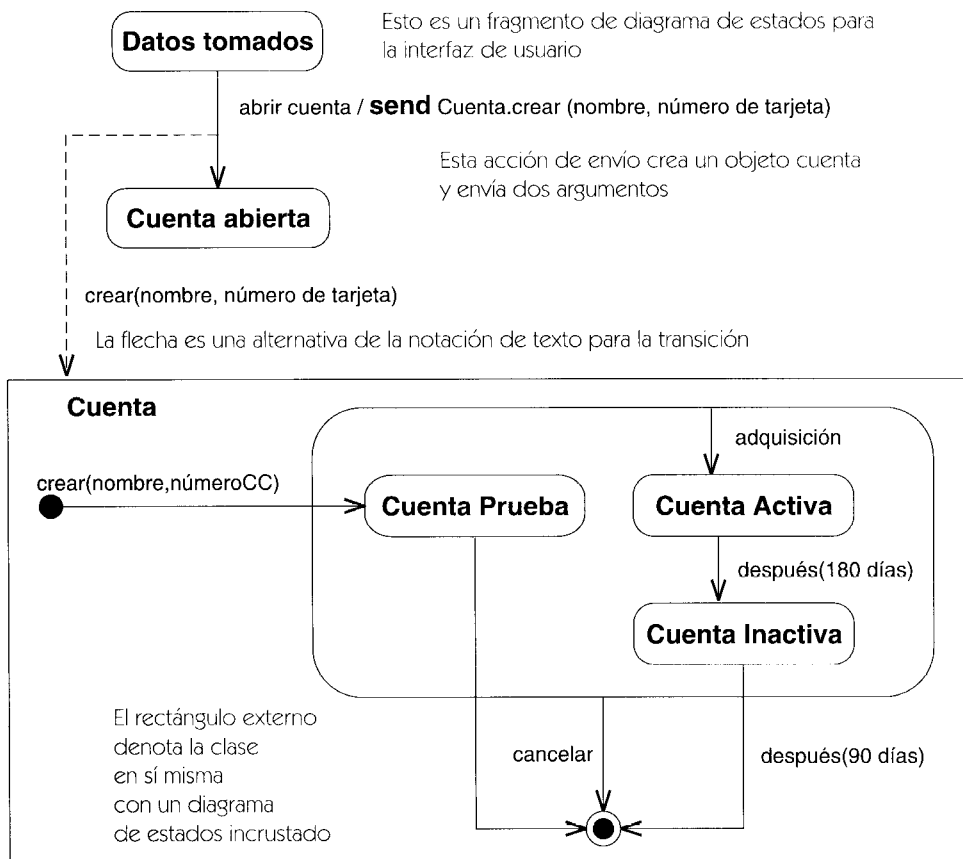


Figura 13.79 Creación de un nuevo objeto enviando un mensaje

Se puede usar este modelo aun cuando el lenguaje destino, como C++, no admita las clases como objetos en tiempo de ejecución. En tal caso, la acción de creación se compila (lo cual impone ciertas restricciones sobre su generalidad, por ejemplo, el nombre de la clase tiene que ser un valor literal) pero la intención subyacente es la misma.

Una dependencia de envío es un estereotipo de una dependencia de uso del emisor de la señal con respecto a la clase que recibe la señal.

Notación de texto

Dentro de una transición, enviar una señal posee su propia sintaxis, aunque en realidad no es más que un caso especial de una acción. Dentro de la secuencia de acción, la expresión de envío tiene la sintaxis

send expresión-destino . nombre-mensaje-destino (argumentos_{lista}.)

La palabra reservada **send** es opcional. Se puede distinguir una llamada de un envío por la declaración del nombre del mensaje. En algunas ocasiones, sin embargo, resulta útil una distinción explícita.

La expresión-destino tiene que producir, al evaluarse, un conjunto de objetos. Es válido un conjunto que contenga un único objeto. El mensaje se enviará a todos los objetos del conjunto.

El nombre-mensaje-destino es el nombre de una señal o de una operación que admitan los objetos destino. Los argumentos son expresiones que al evaluarse producen valores que deben ser compatibles con los parámetros declarados del evento u operación. La diferencia entre una señal y una operación está basada en las declaraciones de señales en el paquete y de operaciones en la clase destino. No hay ambigüedad en el modelo interno.

Ejemplo

Esta transición interna selecciona un objeto situado dentro de una ventana empleando la localización del cursor y después le envía una señal para resaltarlo.

clic-botón-derecho (localización) [localización in ventana]

/objeto:= seleccionar-objeto (localización); **send** objeto.resaltar()

Notación de diagramas

El envío de un mensaje se puede mostrar también mediante símbolos en un diagrama.

El envío de un mensaje entre máquinas de estados se puede mostrar dibujando una flecha discontinua que va del emisor al receptor. La flecha está etiquetada con las expresiones de nombre y argumentos del mensaje. El diagrama de estados tiene que estar contenido en un rectángulo que representa un objeto o clase dentro del sistema. Gráficamente, los diagramas de estados pueden estar anidados físicamente dentro del símbolo de un objeto o bien pueden ser implícitos y ser mostrados en algún otro lugar. Los diagramas de estados representan el control de los objetos que colaboran. Los que interactúan pueden ser roles de una colaboración o bien clases que indican la forma general en que se comunican los objetos de las clases. La Figura 13.80 contiene diagramas de estado que muestran el envío de señales entre tres objetos.

Obsérvese que esta notación también se puede utilizar en otras clases de diagramas para mostrar el envío de eventos entre clases u objetos.

El símbolo del emisor (que se encuentra en el origen de la flecha) puede ser una:

- | | |
|------------|--|
| clase | El mensaje es enviado por un objeto de la clase en algún instante de su vida, pero no se especifican detalles. |
| transición | El mensaje se envía formado parte de la acción de disparo de la transición (Figura 13.79 y Figura 13.80). Se trata de una presentación alternativa de la sintaxis de texto para enviar mensajes. |

El símbolo del receptor (situado en la cabeza de flecha) puede ser una:

- | | |
|-------|--|
| clase | El mensaje es recibido por el objeto y puede desencadenar una transición dentro del objeto. El símbolo de clase puede contener un diagrama de estados (Figura 13.80). El objeto receptor puede poseer múltiples transiciones que utilicen como disparador el mismo evento. Esta notación no se puede emplear cuando el objeto destino se calcula dinámicamente. En tal caso, es preciso utilizar una expresión de texto. |
|-------|--|

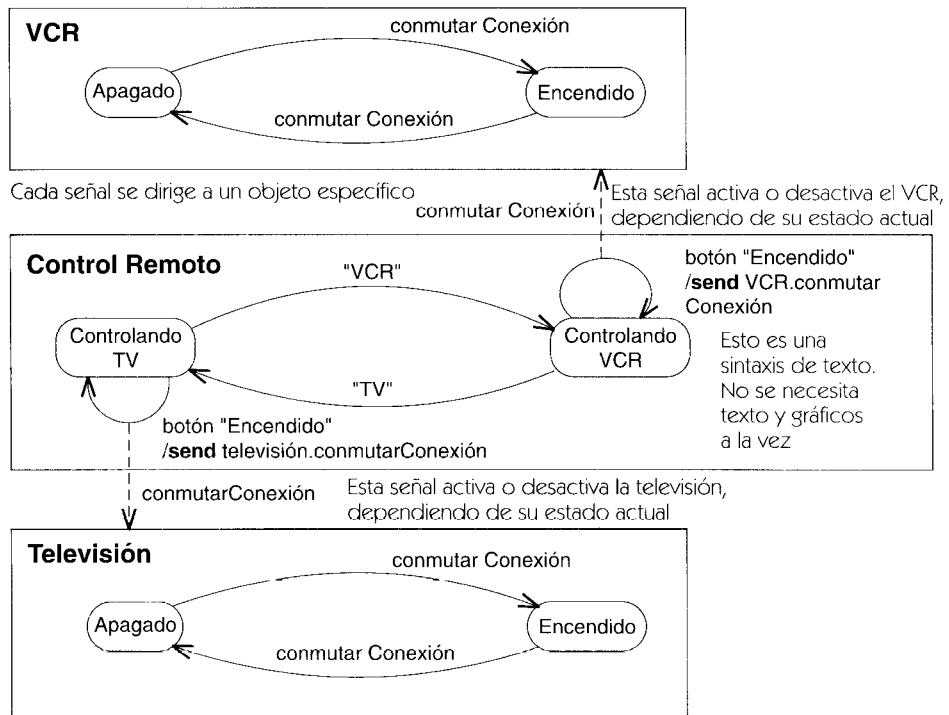


Figura 13.80 Envío de señales entre objetos

metaclass Esta notación se emplearía para modelar la invocación de operaciones con alcance de clase, tales como la creación de una nueva instancia. La recepción de tales mensajes da lugar a la instanciación de un nuevo objeto en su estado inicial por defecto. El evento que ve el receptor puede utilizarse para disparar una transición de su estado inicial por defecto y por tanto representa una forma de pasar información procedente del creador al nuevo objeto.

transición La transición tiene que ser la única transición de la clase que haga uso del evento, o al menos la única transición que pudiera ser disparada por el envío de ese mensaje en particular (Figura 13.79). Esta notación no es posible cuando la transición disparada depende del estado del objeto receptor. En tal caso, la flecha tiene que dibujarse para que llegue hasta la clase.

Dependencia de envío. Una dependencia de envío se muestra como una flecha discontinua que va desde la clase u operación que envía la señal hasta la clase que recibe al señal; se asocia el estereotipo «send» a la flecha.

escenario

Secuencia de acciones que ilustra un comportamiento. Un escenario se puede utilizar para ilustrar la interacción o ejecución de una instancia de caso de uso.

espacio de nombres

Cierta parte del modelo en la cual se pueden definir y usar los nombres. Dentro de un espacio de nombres, cada nombre posee un significado exclusivo.

Semántica

Todos los elementos con nombre se declaran en un espacio de nombres y sus nombres están dentro del alcance así definido. Los espacios de nombres del más alto nivel son los paquetes (incluyendo los subsistemas), contenedores cuyo propósito es agrupar elementos primordialmente para el acceso y comprensión por parte de personas, así como organizar los modelos para su almacenamiento y manipulación por computador durante el desarrollo. Los elementos primarios de los modelos, como las clases, asociaciones, máquinas de estados y colaboraciones, actúan como espacios de nombres para sus contenidos, tales como atributos, extremos de asociación, estados y roles de colaboración. El alcance de cada elemento del modelo se discute dentro de su propia descripción. Todos estos elementos del modelo poseen su propio espacio de nombres distintivo.

Los nombres que se definen dentro de un espacio de nombres tienen que ser únicos (después de todo, ése es su propósito). Dado un espacio de nombres y un nombre, es posible hallar un elemento particular dentro del espacio de nombres (si es que tiene nombre —algunos elementos son anónimos y deben hallarse mediante alguna relación con elementos que tengan nombre—). Es posible buscar hacia el interior a lo largo de una lista de espacios de nombres anidados si se dan sus nombres.

Para poder acceder a otros espacios de nombres, un paquete puede acceder a otro paquete o importarlo.

El propio sistema define el espacio de nombres más externo, que proporciona la base para todos los nombres absolutos. Se trata de un paquete, que normalmente posee paquetes anidados en varios niveles hasta que finalmente se llega a los elementos primitivos.

Notación

La notación de un nombre de ruta, que es una ruta o camino que recorrer varios espacios de nombres anidados, se obtiene por concatenación de los nombres de los espacios de nombres (tales como paquetes o clases) separándolos mediante parejas de signos de dos puntos (::).

InterfazUsuario :: SistemaAyuda :: PantallaAyuda

especialización

Producir una descripción más específica del modelo añadiéndole hijos. La relación opuesta es la generalización, que también se utiliza como nombre de la relación existente entre el elemento más específico y el más general, por cuanto no existe un término adecuado para la relación que carezca de dirección. Un elemento hijo es la especialización de un elemento padre. A la inversa, el padre es la generalización del hijo.

Véase generalización.

especificación

Descripción declarativa de lo que es o hace algo. Por ejemplo, un caso de uso o una interfaz es una especificación. Por contraste: implementación.

especificador de interfaz

Especificación del comportamiento requerido por parte de una clase asociada para satisfacer la intención de la asociación. Consta de la referencia de una interfaz, clase u otro clasificador que especifique el comportamiento requerido.

Véase también rol de asociación, nombre de rol, tipo.

Semántica

En muchos casos, una clase asociada puede tener más funcionalidad de la necesaria para admitir una cierta asociación. Por ejemplo, la clase puede participar en otras asociaciones y su comportamiento global es el comportamiento necesario para admitirlas todas a la vez. Puede ser conveniente especificar con más precisión la funcionalidad que necesita una clase para admitir una cierta asociación. Un especificador de interfaz es un clasificador conectado al final de una asociación, que indica la funcionalidad necesaria para admitir la asociación, sin tener en cuenta la utilización de la clase destino por parte de otras asociaciones.

No se necesita un especificador de interfaz. En muchos casos, incluso en la mayoría, una clase que participa en una asociación posee tan sólo la funcionalidad necesaria y no es preciso decir más. Si se omite un especificador de interfaz, se puede emplear la asociación para acceder a la funcionalidad de la clase asociada.

El especificador puede ser un conjunto de clasificaciones, cada uno de los cuales enuncia el comportamiento que tiene que admitir la clase destino. La clase destino tiene que admitirlos todos, pero puede hacer más que lo que requieran los especificadores.

Notación

Un especificador de interfaz se representan mediante la sintaxis

nombre_de_rol: nombrei_{lista,}

en lugar de un nombre de rol normal, en donde **nombrei** es el nombre de una interfaz o de algún otro clasificador. Si hay más de un especificador, sus nombres se darán en una lista separada por comas.

Ejemplo

En la Figura 13.81, la clase **Servidor** almacena las solicitudes en una clase **Matriz**. Para este propósito, sin embargo, se necesita únicamente la funcionalidad de una clase **Cola**. No se

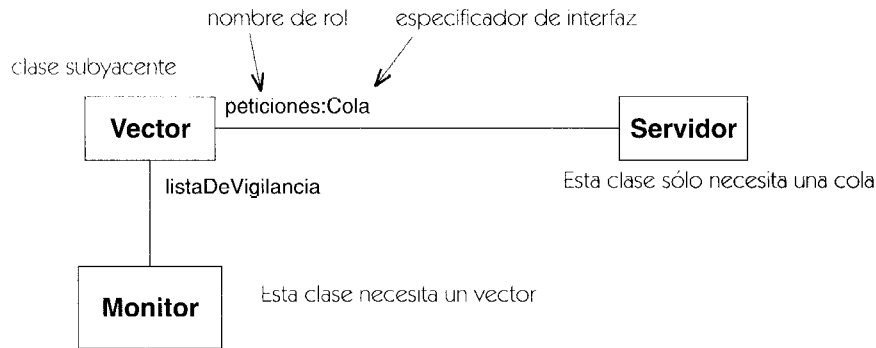


Figura 13.81 Especificador de interfaz

efectúa un acceso aleatorio a la información, por ejemplo. La clase **Matriz** como tal satisface las necesidades del especificador de interfaz **Cola** —una matriz incluye la funcionalidad de una cola—. El **Monitor**, sin embargo, posee una **Matriz** que emplea para visualizar el estado de las solicitudes. El **Monitor** hace uso de la funcionalidad plena de una **Matriz**.

Discusión

El uso de un nombre de rol y de un especificador de interfaz son equivalentes a la creación de una pequeña colaboración que incluya únicamente un rol de asociación y dos roles clasificadores, cuya estructura está definida por el nombre de rol y el rol de clasificador de la asociación original. La asociación y las clases originales son, por tanto, un uso de la colaboración. La clase original tiene que ser compatible con el especificador de interfaz (que puede ser una interfaz o un tipo).

estado

Condición o situación, durante la vida de un objeto, a lo largo de la cual éste satisface alguna condición, realiza una cierta actividad o espera algún evento.

Véase también actividad, grafo de actividades, estado compuesto, acción de entrada, acción de salida, estado final, transición interna, pseudoestado, máquina de estados, submáquina, estado de sincronización, transición.

Semántica

Un objeto almacena una serie de estados a lo largo de su vida. El objeto permanece en un cierto estado durante una cantidad de tiempo finita (no instantánea). Se pueden introducir estados neutros por comodidad, que ejecutan acciones triviales y salen inmediatamente. Pero no es éste el propósito principal de los estados y los estados neutros se pueden eliminar, aunque resultan útiles para evitar duplicaciones.

Los estados están contenidos en una máquina de estados que describe la forma en que evoluciona la historia de un objeto con el paso del tiempo y como respuesta a los eventos. Cada máquina de estados describe el comportamiento de los objetos de una clase. Cada clase puede poseer una máquina de estados. Una transición describe la respuesta de un objeto en un cierto estado frente a la recepción de un evento: el objeto ejecuta una acción opcional asociada a la transición y pasa a un nuevo estado. Cada estado posee su propio conjunto de transiciones.

Las acciones son atómicas y no interrumpibles. Toda acción está asociada a una transición, un cambio de estado, que también es atómico y no interrumpible. La actividad en curso puede estar asociada a un estado. Esta actividad se enuncia como una máquina de estados anidada o una expresión **do**. Alternativamente, la actividad en curso puede estar representada mediante una pareja de acciones, una acción de entrada que da comienzo a la actividad cuando se entra en el estado y una acción de salida que concluye la actividad al salir del estado.

Los estados se pueden agrupar para formar estados compuestos. Toda transición que llega a un estado compuesto afecta a todos los estados que contiene, así que aquellos eventos que afecten de igual modo a muchos subestados pueden ser manejados con una sola transición. Un estado compuesto puede ser secuencial o concurrente. En un estado compuesto secuencial, sólo está activo un subestado en cada instante. En un estado compuesto concurrente, todos los subestados están activos simultáneamente.

Para impulsar el encapsulamiento, un estado puede poseer estados iniciales y estados finales. Se trata de pseudoestados, cuyo propósito es ayuda a estructurar la máquina de estados. Una transición a un estado compuesto representa una transición a su estado inicial. Es equivalente a hacer una transición directa al estado inicial, pero el estado se puede utilizar externamente sin tener conocimiento de su estructura interna.

Una transición a un estado final dentro de un estado compuesto representa la terminación de la actividad en el estado que lo encierra. La terminación de la actividad en el estado que lo encierra dispara un suceso de terminación de actividad y hace que se produzca una transición de finalización en ese estado. Una transición de finalización es la transición que carece de un evento disparador explícito (o más exactamente, una transición en que el evento de finalización es el disparador implícito aunque no se haya modelado explícitamente). La finalización del estado más externo de un objeto corresponde a su muerte.

Si un estado es un estado compuesto concurrente, entonces todas las subregiones concurrentes tienen que terminar antes de que se produzca el evento de finalización en el estado compuesto. En otras palabras, una transición de finalización procedente de un estado compuesto concurrente representa una unión de control de todos sus subhilos concurrentes. Espera a que hayan finalizado todos ellos antes de seguir adelante.

Estructura

Los estados poseen las partes siguientes.

Nombre. El nombre del estado, que tiene que ser único dentro del estado que lo contiene. Se puede omitir el nombre, dando lugar a un estado anónimo. El número de estados anónimos que pueden coexistir no está limitado. Un estado anidado puede identificarse mediante su nombre de ruta (si tienen nombre todos los estados en que esté incluido).

Subestados. Si una máquina de estados posee una subestructura anidada, se denomina estado compuesto. Un estado compuesto es o bien una red de subestados disjuntos (esto es, subestados que se activan secuencialmente) o bien un conjunto de subestados concurrentes (esto es, subestados que están todos activados concurrentemente). Un estado que no posee subestructura (salvo las posibles acciones internas) recibe el nombre de estado simple.

Acciones de entrada y salida. Un estado puede tener una acción de entrada y una acción de salida. El propósito de estas acciones es encapsular el estado de tal modo que se pueda utilizar externamente sin tener conocimiento de su estructura interna. La acción de entrada se ejecuta cuando se entra en el estado, después de cualquier posible acción asociada a la transición entrante y antes de cualquier otra actividad interna. La acción de salida se ejecuta al salir del estado, una vez finalizada toda actividad interna y antes de ejecutar cualquier acción que pueda estar asociada a la transición saliente. En una transición que cruce los límites de varios estados, se pueden ejecutar varias acciones de entrada y salida de forma anidada. En primer lugar, se ejecutan las acciones de salida, comenzando por el estado más interno y avanzando hacia el más externo; a continuación se ejecuta la acción asociada a la transición y después las acciones de entrada, comenzando por la más externa y terminando en la más interna. La Figura 13.82 muestra el resultado de disparar una transición que cruza los límites de un estado. Las acciones de entrada y de salida no se pueden evitar en modo alguno, incluyendo la aparición de excepciones. Proporcionan un mecanismo de encapsulamiento para la especificación del comportamiento de la máquina de estados, con la garantía de que se ejecutarán las acciones necesarias en cualquier circunstancia.

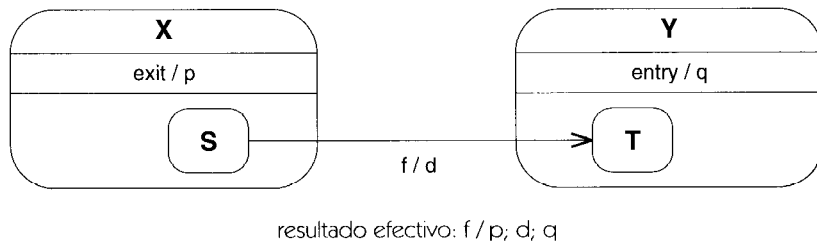


Figura 13.82 Transición a través de los límites de un estado, con acciones de entrada y salida

Actividad interna. Un estado puede contener una actividad interna descrita en forma de una expresión. Cuando se entra en ese estado, la actividad comienza una vez finalizada la acción de entrada. Si termina la actividad, el estado ha finalizado. Entonces se dispara una transición de finalización que sale del estado. En caso contrario, el estado espera al disparo de una transición que dé lugar a un cambio de estado. Si se dispara una transición mientras se está ejecutando la actividad, entonces finaliza la actividad y se ejecuta la acción de salida del estado.

Transiciones internas. Un estado puede poseer una lista de transiciones internas, que son como transiciones normales salvo que no tienen estado destino y no dan lugar a un cambio de estado. Si su evento se produce mientras un objeto se encuentra en el estado que posee la transición, entonces se ejecuta la acción de la transición interna pero no se produce un cambio de estado ni se ejecutan las acciones de entrada ni de salida, aun cuando la transición interna esté declarada en un estado que contiene al actual (porque el estado actual no ha cambiado). Esto es

lo que las distingue de una autotransición, en la cual se produce una transición externa de un estado a ese mismo estado, dando lugar a la ejecución de las acciones de salida de todos los estados anidados dentro del estado que tiene la autotransición, a la ejecución de su propia acción de salida y a la ejecución de su acción de entrada. Las acciones se ejecutan, incluso, en una autotransición al estado actual, del cual se sale y en el que se vuelve a entrar. Si se dispara una autotransición en un estado que contiene al actual, entonces el estado final es el estado contenedor, no el estado actual. En otras palabras, una autotransición puede forzar una salida de un estado anidado, pero no una transición interna.

Submáquina. El cuerpo de un estado puede representar una copia de otra máquina de estados distinta, a la cual se hace referencia por su nombre. La máquina de estados a la que se hace referencia se denomina submáquina porque está anidada dentro de la máquina de estados mayor y el estado que hace la referencia recibe el nombre de estado de submáquina. Una submáquina puede asociarse a una clase que le proporcione el contexto para las acciones que contiene, tales como los atributos que se puedan leer o escribir. Las submáquinas están destinadas a ser reutilizadas en cualquier máquina de estados para evitar la repetición de un mismo fragmento de máquina de estados. Una submáquina es algo parecido a una subrutina de una máquina de estados.

Dentro del estado de referencia a la submáquina, se hace referencia a la submáquina mediante un nombre con una posible lista de argumentos. El nombre tiene que ser el nombre de una máquina de estados que posea un estado inicial y un estado final. Si la submáquina tiene parámetros en su transición inicial, entonces la lista de argumentos tiene que tener argumentos coincidentes con éstos. Cuando se entra en el estado de submáquina, se ejecuta primero su acción de entrada y después comienza la ejecución de la submáquina en su estado inicial. Cuando la submáquina alcanza su estado final, se ejecutan las posibles acciones de salida del estado de submáquina. Entonces se considera que el estado de submáquina está terminado y se puede efectuar una transición basada en la finalización implícita de la actividad.

Una transición a un estado de referencia de una submáquina activa el estado inicial de la submáquina destino. Pero en algunas ocasiones se desea hacer una transición a un estado diferente de la submáquina. Un estado abreviado externo es un pseudoestado que se encuentra dentro de un estado de referencia a la submáquina y que identifica a un estado contenido en la submáquina. Las transiciones se pueden conectar al estado abreviado externo desde otros estados de la máquina de estados principal. Si se dispara una transición a un estado abreviado externo, el estado al que se hace referencia en la copia de la submáquina se activa.

Una submáquina representa una actividad anidada e interrumpible dentro de un estado. Es equivalente a reemplazar el estado de la submáquina por una copia única de la submáquina. En lugar de proporcionar una máquina de estados, se puede asociar a la submáquina una expresión procedural (que es una actividad). Se puede considerar que una actividad define una serie de estados, uno por cada expresión primitiva, que se puede interrumpir entre dos pasos cualesquiera. No es lo mismo que una acción, que es atómica y no se puede interrumpir.

Concurrencia dinámica. Un estado de actividad o un estado de submáquina puede tener una multiplicidad y una expresión de concurrencia. La multiplicidad especifica cuántas copias del estado se pueden ejecutar concurrentemente. El caso normal es una multiplicidad de exactamente uno, lo cual significa que el estado representa un hilo normal de control. Si no se determina el valor de la multiplicidad, el número de unidades de ejecución se determinará

dinámicamente en el tiempo de ejecución. Por ejemplo, el valor 1..5 significa que se ejecutan concurrentemente entre una y cinco copias de esa actividad. Si existe la expresión de concurrencia (que es obligatoria si la multiplicidad no es exactamente uno), entonces en el tiempo de ejecución tendrá que evaluarse como un conjunto de listas de argumentos. La cardinalidad del conjunto indica el número de activaciones concurrentes del estado. Cada estado recibe una lista distinta de valores de argumentos como valor del evento implícito actual. Cuando han finalizado todas las ejecuciones, el estado dinámicamente concurrente se considera finalizado y la ejecución pasa al estado siguiente.

Esta capacidad está destinada a los diagramas de actividades.

Eventos diferibles. Lista de eventos cuya aparición en el estado queda pospuesta, si no disparan una transición, hasta que disparen una transición o el sistema haga una transición a un estado en el que no se puedan retrasar, momento en el cual serán tratados. La implementación de tales eventos diferibles implicaría una cola interna de eventos.

Notación

Un estado se representa mediante un rectángulo con esquinas redondeadas. Puede tener uno o más compartimentos. Los compartimentos son opcionales. Se pueden incluir los siguientes compartimentos:

Compartimento de nombre. Contiene el nombre (opcional) del estado en forma de cadena. Los estados sin nombre son anónimos y son todos diferentes. No es conveniente repetir dos veces el mismo símbolo de estado con nombre en el mismo diagrama, porque puede inducir a confusión.

Estado anidado. Muestra un diagrama de estados de un estado compuesto, formado a su vez por estados subordinados anidados. El diagrama de estados se dibuja dentro de los límites del estado exterior. Las transiciones se pueden conectar directamente a los estados anidados, así como al límite del estado exterior. En una región disjunta, los subestados se dibujan directamente dentro del estado compuesto. En una región concurrente, el símbolo de estado se divide en subregiones mediante líneas discontinuas (esto es, se apilan).

Véase estado compuesto para más detalles y ejemplos.

Compartimento de transición interna. Contiene una lista de acciones o actividades internas que se ejecutan como respuesta a eventos recibidos mientras el objeto se encuentra en ese estado, sin cambiar de estado. Una transición interna tiene el formato

$$\text{nombre-evento}_{\text{opt}} (\text{argumentos}_{\text{lista, opt}}) [\text{condición-guarda}]_{\text{cond, opt}} / \text{expresión-acción}_{\text{opt}}$$

Las expresiones de acción pueden emplear atributos y enlaces del objeto poseedor y parámetros de transiciones entrantes (si aparecen en todas las transiciones entrantes).

La lista de argumentos (incluyendo los paréntesis) se puede omitir si no hay parámetros. La condición de guarda (incluyendo los corchetes) y la expresión de acción (incluyendo la barra) son opcionales.

Las acciones de entrada y de salida tienen la misma forma pero emplean las palabras reservadas **entry** y **exit** que no se pueden emplear para nombres de eventos.

entry / expresión-acción

exit / expresión-acción

Las acciones de entrada y de salida pueden no tener argumentos o condiciones de guarda (porque son invocadas implícitamente y no explícitamente). Para obtener los parámetros de una acción de entrada, se puede acceder al evento actual desde la acción. Esto resulta especialmente útil para obtener los parámetros de creación desde un objeto nuevo.

El nombre de acción reservado **defer** indica un evento que se puede retrasar en un cierto estado y en sus subestados. La transición interna no debe tener condición de guarda ni acciones.

nombre-evento / defer

La palabra reservada **do** representa una expresión para una actividad no atómica.

do / expresión-actividad

Estado de referencia a submáquina. La invocación de una submáquina anidada se muestra mediante una cadena de la forma siguiente situada en el cuerpo del símbolo de estado:

include nombre-maquina(argumentos_{lista})_{opt}

Tanto las actividades **do** como las submáquinas describen cómputos no atómicos que normalmente se ejecutan hasta haber finalizado, pero que pueden ser absorbidos por un evento que dispare una transición.

Ejemplo

La Figura 13.83 muestra un estado con transiciones internas. La Figura 13.84 muestra la declaración y utilización de una submáquina.

Concurrencia dinámica. La concurrencia dinámica con un valor distinto de uno exactamente se muestra mediante una cadena de multiplicidad situada en la parte superior derecha del símbolo de estado. La cadena no debería incluirse para la ejecución secuencial normal. Esta notación está destinada principalmente a los diagramas de actividades y debe evitarse, en general, en los diagramas de estados.

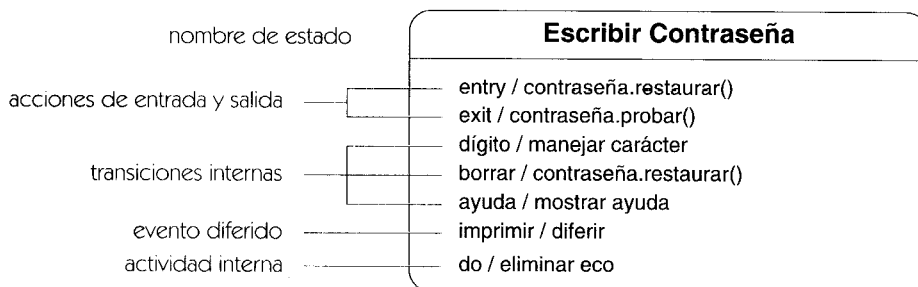


Figura 13.83 Transiciones internas, con acciones de entrada y salida y un evento diferido

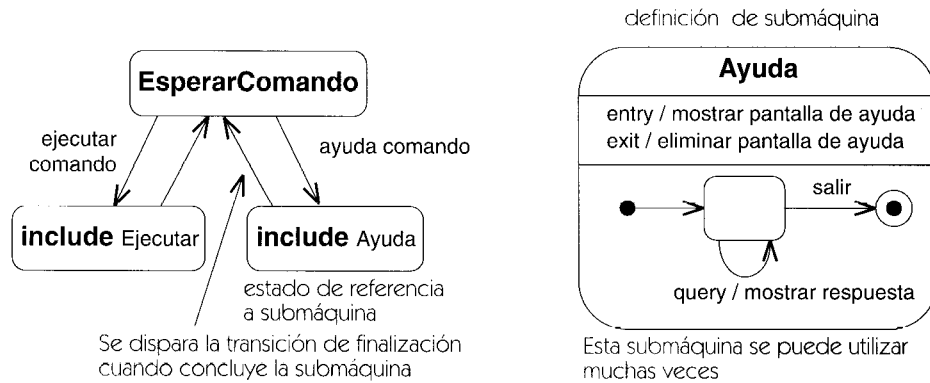


Figura 13.84 Submáquina

estado abreviado externo

Pseudocódigo dentro de un estado de referencia a submáquina que identifica a un estado de la submáquina a la que se hace referencia.

Véase también transición abreviada, submáquina, estado de referencia a submáquina.

Semántica

Una transición a un estado de referencia a submáquina activa el estado inicial de la submáquina. Pero en algunas ocasiones se desea una transición a un estado diferente de la submáquina. Se sitúa el estado externo dentro de un estado de referencia a submáquina y mediante éste se identifica a un estado perteneciente a la submáquina. Se pueden conectar transiciones entre el estado abreviado externo y otros estados pertenecientes a la máquina de estados que lo contiene. Si se dispara una transición a un estado abreviado externo, el estado identificado dentro de la submáquina se activa. Si un estado de la submáquina está activado, entonces una transición procedente de un estado abreviado externo que lo identifique es candidata a ser disparada. No están permitidas las conexiones entre estados abreviados externos del mismo estado de referencia a submáquina.

Notación

Una transición desde o hacia un estado abreviado externo se dibuja como una transición abreviada hacia o desde el estado de referencia a submáquina —esto es, como una flecha que acaba o empieza en una barra corta dentro del símbolo de estado correspondiente al estado de referencia a submáquina—. La barra está etiquetada con un nombre, que tiene que coincidir con el estado de la submáquina a que se hace referencia.

La Figura 13.91 muestra una abreviatura dentro de un estado de referencia a subrutina. La Figura 13.92 muestra la definición de submáquina correspondiente.

estado destino

El estado de la máquina de estados al que se llega al dispararse la transición. Una vez que un objeto maneja un evento que da lugar a que se dispare una transición, el objeto queda en el

estado destino de la transición (o estados destino, si se trata de una transición compleja con múltiples estados destino). No es aplicable a una transición interna, que no da lugar a cambios de estado.

Véase transición.

estado compuesto

Un estado que consiste en subestados concurrentes (ortogonales) o subestados secuenciales (disjuntos).

Véase también transición compleja, estado simple, estado.

Semántica

Un estado compuesto puede descomponerse en subestados concurrentes usando relaciones AND, y en estados mutuamente disjuntos usando relaciones OR. Un estado se puede refinar solamente de una de estas dos maneras. Sus subestados se pueden refinar de cualquier manera. Si un estado compuesto secuencial está activo, uno de sus subestados disjuntos es activo. Si un estado compuesto concurrente está activo, todos sus subestados ortogonales están activos. El efecto final es un árbol Y-O. Cada máquina de estados tiene un estado de nivel superior, que es un estado compuesto.

Un sistema puede estar en múltiples estados simultáneamente. El conjunto de estados activos se llama configuración del estado activo. Si un estado anidado está activo, entonces todos los estados compuestos que lo contienen están activos. Si el objeto permite concurrencia, entonces puede estar activo más de un subestado concurrente.

Véase transición compleja para una discusión de la ejecución concurrente. La Figura 13.4 muestra un árbol Y-O.

Un objeto creado por primera vez comienza en su estado inicial, que el estado compuesto exterior debe tener. El evento que crea el objeto se puede utilizar para accionar una transición del estado inicial. Los argumentos del evento de creación están disponibles para esta transición inicial.

Un objeto que cambia a su estado final exterior es destruido y deja de existir.

Estructura

Un estado compuesto contiene un conjunto de subestados. Un estado compuesto es concurrente o secuencial.

Un estado compuesto secuencial debe tener, en la mayoría de los casos, un estado inicial y un estado final. También suele tener, en la mayoría de los casos, un estado superficial de historia y un estado profundo de historia.

Un estado compuesto concurrente no puede tener un estado inicial, un estado final, o estados de historia, pero cualquier estado compuesto secuencial anidado dentro de ellos puede tener tales pseudoestados.

Notación

Un estado compuesto es un estado con detalle subordinado. Tiene un compartimento para el nombre, un compartimento de transición interna, y un compartimento gráfico que contiene un diagrama anidado que muestra el detalle subordinado. Todos los compartimentos son opcionales. Por conveniencia y el aspecto, los compartimentos de texto (nombre y transiciones internas) se pueden contraer como lengüetas (tabs) dentro de la región gráfica, en vez de atravesarla horizontalmente.

Una expansión de un estado concurrente compuesto, en subestados concurrentes se muestra dividiendo el compartimento gráfico del estado usando líneas discontinuas para separarlo en subregiones. Cada subregión es un subestado concurrente, el cual puede tener un nombre opcional y debe contener un diagrama de estados anidado con subestados disjuntos. Los compartimentos de texto del estado completo se separan de los subestados concurrentes por una línea continua.

Una expansión de un estado en subestados disjuntos se muestra mediante un diagrama de estados anidado dentro de la región gráfica.

Un estado inicial se representa con un círculo negro pequeño. En una máquina de estados de nivel superior, la transición de un estado inicial se puede etiquetar con el evento que crea el objeto. Si no, debe estar sin etiqueta. Si está sin etiqueta, representa cualquier transición al estado que lo incluye. La transición inicial puede tener una acción. El estado inicial es un dispositivo notacional. Un objeto no puede estar en tal estado, pero debe pasar desde él a un estado real.

Un estado final se representa como una circunferencia que rodea un círculo sólido pequeño (un ojo de buey). Representa la terminación de la actividad en el estado que lo incluye, y acciona una transición en el estado que lo incluye, etiquetada por el evento implícito de terminación de actividad (representado generalmente como una transición sin etiqueta).

Ejemplo

La Figura 13.85 muestra un estado compuesto secuencial que contiene dos subestados disjuntos, un estado inicial, y un estado final. Cuando el estado compuesto llega a ser activo, el subestado **Comienzo** (destino del estado inicial) se activa primero.

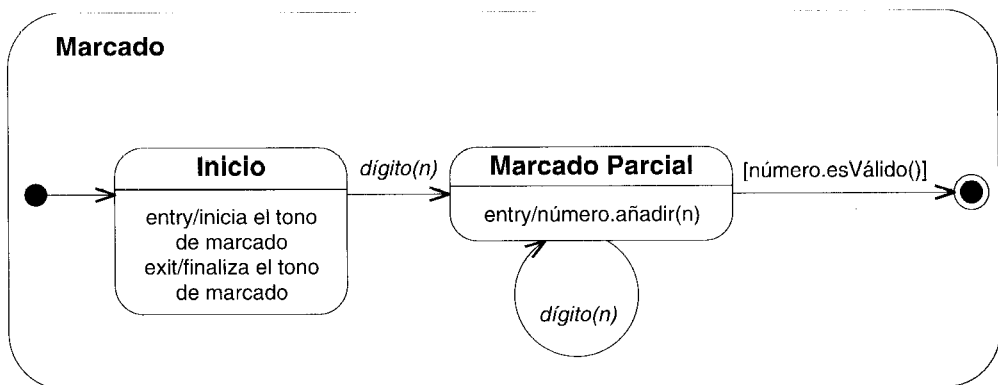


Figura 13.85 Estado compuesto secuencial

La Figura 13.86 muestra un estado compuesto concurrente que contiene tres subestados ortogonales. Cada subestado concurrente se descompone más a fondo en subestados secuenciales. Cuando el estado compuesto **Incompleto** llega a ser activo, los destinos de los estados iniciales se activan. Cuando las tres subregiones alcanzan el estado final, entonces se dispara la transición de la terminación del estado compuesto externo **Incompleto** y el estado **Aprobado** pasa a estar activo. Si ocurre el evento de **Suspense** mientras el estado **Incompleto** está activo, entonces las tres subregiones concurrentes terminan y el estado **Suspense** se convierte en activo.

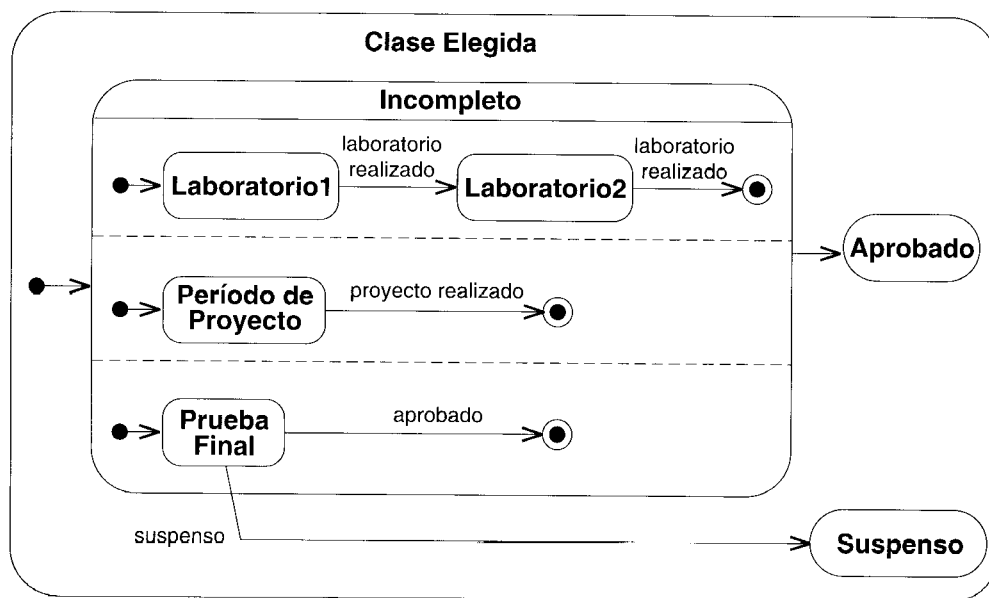


Figura 13.86 Estado concurrente compuesto

estado de acción

Estado cuyo propósito es ejecutar una acción y luego realizar una transición para cambiar a otro estado.

Véase también estado de actividad, transición de finalización.

Semántica

Un estado de acción es un estado cuyo propósito es ejecutar una acción de entrada, después de la cual realiza una transición de finalización hacia otro estado. El estado de acción es atómico, lo que significa que no puede ser terminado por una transición de un evento externo. Conceptualmente representa una computación que se completa de modo casi instantáneo y que no interactúa con otras acciones que se producen al mismo tiempo. En la práctica, puede requerir tiempo para completarse, pero será siempre menor que el tiempo de respuesta requerido por cualquier evento que pudiera ocurrir. No puede tener transiciones disparadas por eventos. Un estado de acción no tiene subestructura, acciones internas ni transiciones internas. Es una

especie de estado tanto útil para organizar máquinas de estados en estructuras lógicas. Normalmente cuenta con una transición de finalización saliente, aunque puede tener múltiples transiciones de finalización salientes si tienen condiciones de guarda (y por tanto, cada una de ellas representa una bifurcación).

Notación

No hay una notación especial para los estados de acción, por lo que se representan como un estado ordinario con una acción de entrada, aunque también pueden representarse mediante un estado de actividad.

estado de actividad

Estado que representa la ejecución de un cómputo puro que tiene una subestructura, normalmente la invocación a una operación, una sentencia interna a ella, o la realización de un procedimiento del mundo real. Un estado de actividad puede ser terminado externamente por un evento que fuerce una transición de salida del estado. Un estado de actividad no tiene por qué terminar por sí mismo, pues no existe ningún límite sobre cuánto tiempo puede estar activo.

Véase también actividad, transición de finalización.

Semántica

Un estado de actividad es un estado con un cómputo interno, y normalmente, al menos una transición de finalización que se dispare a la finalización de la actividad del estado (puede haber varias transiciones de este tipo si tienen condiciones de guarda). Los estados de actividad no deberían tener transiciones internas ni transiciones de salida basadas en eventos explícitos: utilice estados normales para modelar estas situaciones. El uso habitual de un estado de actividad es modelar un paso en la ejecución de un algoritmo (un procedimiento). Si todos los estados de un modelo son estados de actividad y las actividades concurrentes no acceden a los mismos valores, el cómputo es determinista aunque implique ejecución concurrente.

Un estado de actividad puede hacer referencia a una submáquina, normalmente a otro grafo de actividades. Esto es equivalente a expandir una copia de la red de la submáquina referenciada en el lugar que ocupa el estado de actividad. Es una subrutina de máquina de estados.

Un estado de actividad es un estado del proceso de ejecución de un procedimiento en lugar de un estado de un objeto normal.

Un estado de acción es un estado de actividad atómico, es decir, no puede ser interrumpido por una transición mientras permanece activo. Puede modelarse como un estado de actividad con una única acción de entrada.

Los estados de actividad se pueden utilizar en las máquinas de estados ordinarias, pero es más común utilizarlos en los grafos de actividades.

Las transiciones que salen de un estado de actividad normalmente no deberían incluir un evento disparador, ya que dichas transiciones se disparan implícitamente cuando termina la actividad del estado. Las transiciones pueden incluir condiciones de guarda y acciones. Asegúrese de que todas las posibles condiciones están cubiertas en las transiciones que salen de una actividad, pues de lo contrario el control podría quedar colgado. Si se evalúa a verdadero más de una condición de guarda, no estará definida cuál de ellas debe escogerse. Puede ser una elección no determinista, o imponerse una regla para cambiar la semántica.

En otras situaciones utilice un estado normal.

Notación

Un estado de actividad se representa como se muestra en la Figura 13.87, con una figura en forma de píldora (un símbolo con líneas horizontales arriba y abajo y lados convexos), dentro de la cual se escribe la expresión de actividad. La expresión de actividad no tiene por qué ser única en el diagrama.



Figura 13.87 Actividades

Discusión

Los estados de acción están pensados para operaciones cortas de contabilidad y los estados de actividad para cálculos de cualquier duración o complejidad. Una acción podría bloquear el sistema, por lo que debe ser breve, mientras que una actividad puede ser finalizada externamente, por lo que el sistema no necesita esperar a que finalice si ocurre algo urgente. La semántica de UML no impide acciones prolongadas, pero un generador de código debería asumir que una acción tiene que completarse de una sola vez, mientras que una actividad puede ser interrumpida por otras acciones.

estado de flujo de objeto

Se trata de un estado que representa la existencia de un objeto o de una clase particular en cierto punto de una computación, tal como una vista de interacción o un grafo de actividades.

Véase también flujo de objeto, clase-en-un-estado.

Semántica

Tanto los grafos de actividades como las vistas de interacción representan el flujo de objeto entre operaciones de objetos destino por medio de mensajes, pero los mensajes no muestran el

flujo de los objetos que son los argumentos de esas operaciones. Este tipo de flujo de información se puede representar en modelos de comportamiento empleando estados de flujo de control.

Un estado de flujo de objeto representa un objeto de una clase que existe en un cierto punto dentro de una computación, tal como un grafo de actividades o una vista de interacción. El objeto puede ser la salida de una actividad y la entrada de otras muchas actividades. En un grafo de actividades, puede ser el destino de una transición (con frecuencia se trata de una bifurcación y la otra rama es la ruta principal de control); puede ser el origen de una transición de finalización a una actividad. Cuando se dispara la transición precedente, el estado de flujo de objeto se activa. Esto representa la creación de un objeto de esa clase. Para mostrar el paso de un objeto a un estado, mejor que la creación de un nuevo objeto, se puede declarar un estado de flujo de objeto como una clase en cierto estado, una clase-en-un-estado.

El estado de flujo de objeto tiene que coincidir con el tipo del resultado o parámetro que representa. Si es la salida de una operación, tiene que satisfacer el tipo del resultado. Si es la entrada de una operación, tiene que coincidir con el tipo de un parámetro.

Si el estado de flujo de objeto va seguido por una transición de finalización a una actividad, entonces se puede realizar la actividad en cuanto esté disponible el valor del objeto. No se necesita ninguna entrada de control adicional. En otras palabras, la creación de datos en la forma correcta es el disparador para llevar a cabo la actividad.

Para mostrar que una actividad requiere tanto una ruta de control como la presencia de un valor, la acción anterior de la ruta de control y un estado de flujo de objeto para ese valor pueden llevar a una transición compleja. Se efectúa la actividad cuando están preparadas todas las transiciones de entrada. Las rutas múltiples hacia una transición indican sincronización.

Los estados de flujo de objeto suelen ser útiles para documentar relaciones de entrada-salida a efectos de comprensión humana, más que para especificar con precisión una computación. La información mostrada por los estados de flujo de objeto ya está disponible.

La producción de un evento por parte de una actividad en un grafo de actividades se puede modelar como un estado de flujo de objeto cuyo clasificador es una señal. Se puede emplear el estereotipo «signal». El estado de flujo de objeto es una salida de la actividad. Si la actividad produce múltiples eventos, entonces los estados de control de flujo son los destinos de una bifurcación.

Notación

Un objeto de una clase en un cierto estado se muestra en un diagrama de actividades mediante un rectángulo que contiene el nombre de la clase subrayado seguido por el nombre del estado entre corchetes cuadrados

nombre de la clase [nombre del estado]

Un ejemplo podría ser

Pedido [Efectuado]

Un símbolo de flujo de objetos representa la existencia del objeto en el estado del procedimiento en sí y no simplemente el propio objeto, como datos. El símbolo de flujo de objetos (que representa un estado) puede aparecer como destino de una flecha de transición y como origen de múltiples flechas de transición. Para distinguirlas de las transiciones ordinarias en un diagrama de actividades, se dibujan como flechas discontinuas en lugar de emplear flechas continuas. Representan el flujo de objetos.

Ejemplo

La Figura 13.88 muestra estados de control de flujo en un diagrama de actividades. Se crea un estado de flujo de objeto cuando finaliza una operación. Por ejemplo, **Pedido[Efectuado]** se crea cuando termina **SolicitarServicio**. Como esta actividad va seguida por otra actividad, el estado

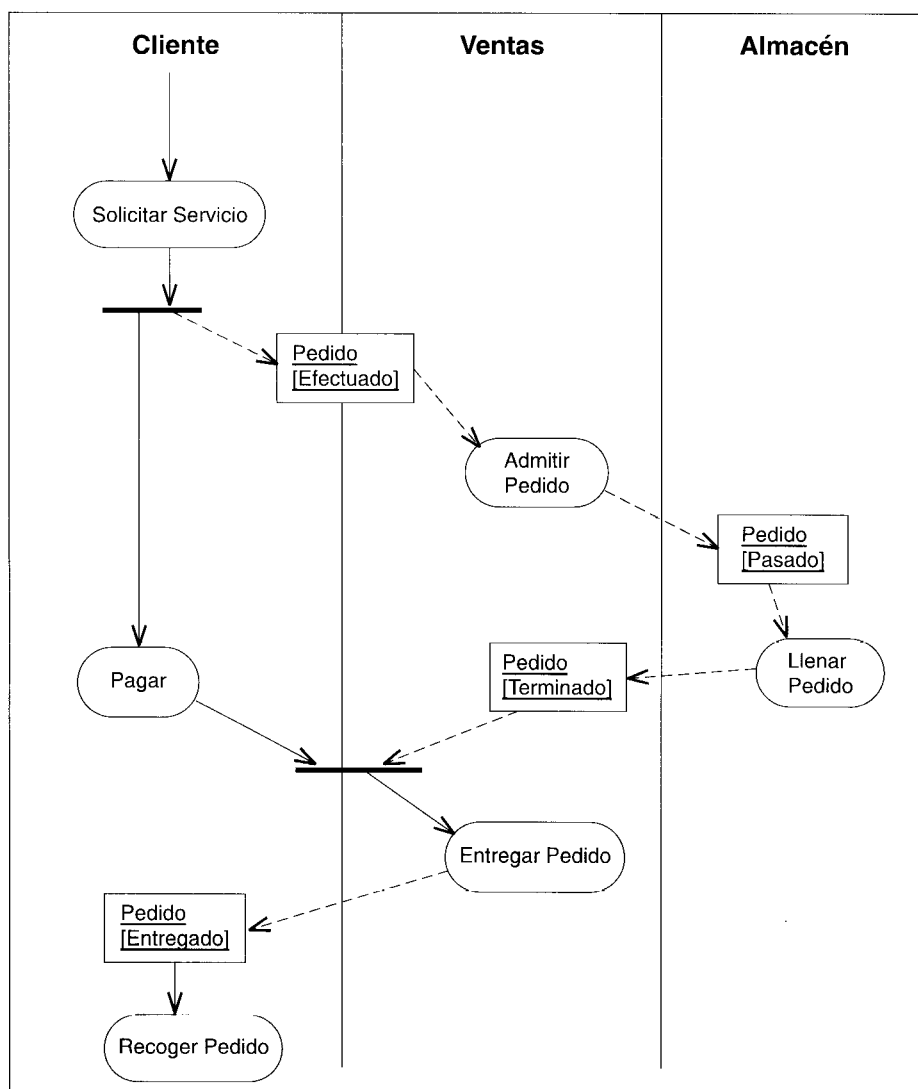


Figura 13.88 Estados de flujo de objetos en un diagrama de actividades

de flujo de objeto **Pedido[Efectuado]** es una salida de un símbolo de bifurcación. El estado **Pedido[Pasado]**, por otra parte, es el resultado de finalizar la actividad **AdmitirPedido**, que no posee otras actividades descendientes.

La Figura 13.89 muestra una parte de un diagrama de actividades dedicado a la construcción de una casa. Una vez construida la estructura, el carpintero ya puede trabajar en el tejado y la casa está preparada para instalar la fontanería. Estos eventos se modelan como estados de control de flujo de señales —**Carpintero libre** y **Estructura**—. Como resultado de estos eventos, se puede construir el tejado y se pueden instalar las tuberías. Por tanto, los estados de control de flujo se muestran como entradas de las actividades. En el modelo, la producción de un suceso por haber terminado una actividad y su utilización como disparador de la siguiente actividad están implícitos en la conexión de las actividades. La necesidad de un evento manifiesto se ha omitido. Por tanto, la aparición de las señales como estados de control de flujo es por estructura de información, más que de implementación.

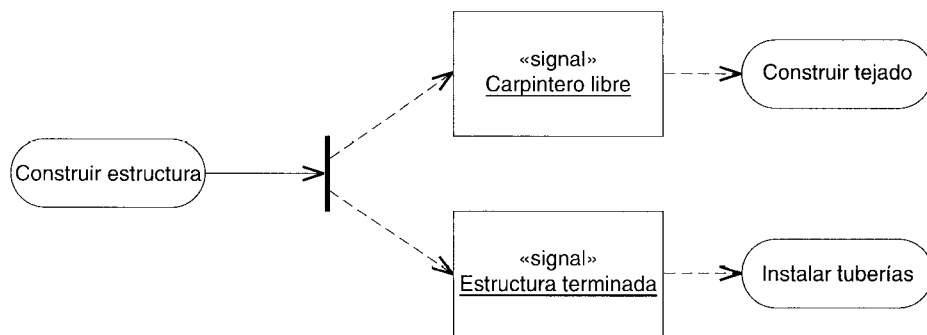


Figura 13.89 Producción de señales en un diagrama de actividades

Discusión

Un estado de flujo de objeto representa el punto de vista de flujo de datos de una computación. A diferencia del flujo de datos tradicional, sin embargo, existe en un punto definido dentro de un modelo de control de flujo (una máquina de estados o un grafo de actividades) y no dentro de un modelo de flujo de datos. Esto lo pone directamente en un marco de trabajo orientado a objetos. La orientación a objetos une los puntos de vista de la estructura de datos, el flujo de objeto y el flujo de datos en un único modelo.

estado de historia

Un pseudoestado que indica que el estado compuesto que lo contiene recuerda su subestado activo previo una vez que concluye.

Véase también estado compuesto, pseudoestado, máquina de estados, transición.

Semántica

El estado de historia permite que un estado compuesto secuencial recuerde el último subestado que haya estado activado en él antes de una transición procedente del estado compuesto. Una

transición al estado de historia da lugar a que vuelva a activarse de nuevo el subestado anteriormente activo. Se llevan a cabo todas las acciones de entrada necesarias. Un estado de historia puede tener transiciones entrantes que procedan de fuera del estado compuesto o del estado inicial. Un estado de historia puede tener una sola transición saliente sin rotular. Esta transición indica el estado de historia almacenado inicialmente. Se hace uso de él si una transición llega al estado de historia cuando no está presente ningún estado almacenado. El estado de historia no puede tener transiciones entrantes que procedan de otros estados pertenecientes al estado compuesto porque ya está activo.

El estado de historia puede recordar una *historia próxima* o una *historia profunda*. Un estado de historia próxima recuerda y reactiva un estado con el mismo nivel de anidamiento que el estado de historia en sí. Si una transición procedente de un subestado ha salido directamente del estado compuesto, se activa el subestado que lo contiene y que ocupa el nivel más elevado dentro del estado compuesto. Un estado de historia profunda recuerda un estado que puede haber estado anidado a alguna profundidad dentro del estado compuesto. Para recordar un estado profundo, la transición debe haber sacado directamente del estado profundo al estado compuesto. Si una transición pasa de un estado profundo a otro más superficial, que después hace una transición y sale del estado compuesto, entonces el estado que se recordará será el menos profundo. Una transición a un estado de historia profunda restaura el estado anteriormente activo sea cual fuere la profundidad. En el proceso, se ejecutan las acciones de entrada si están presentes en estados internos que contengan el estado recordado. Un estado compuesto puede tener a la vez un estado de historia próximo y un estado de historia profundo. Tiene que haber una transición entrante que esté conectada a uno o al otro.

Si un estado compuesto alcanza su estado final, entonces pierde la historia que pudiera tener almacenada y se comporta como si no se hubiera entrado en él por primera vez.

Notación

Un estado de historia próxima se representa mediante un pequeño círculo que contiene la letra **H**, tal como se muestra en la Figura 13.90. Un estado de historia profunda se representa mediante un círculo que contiene **H***.

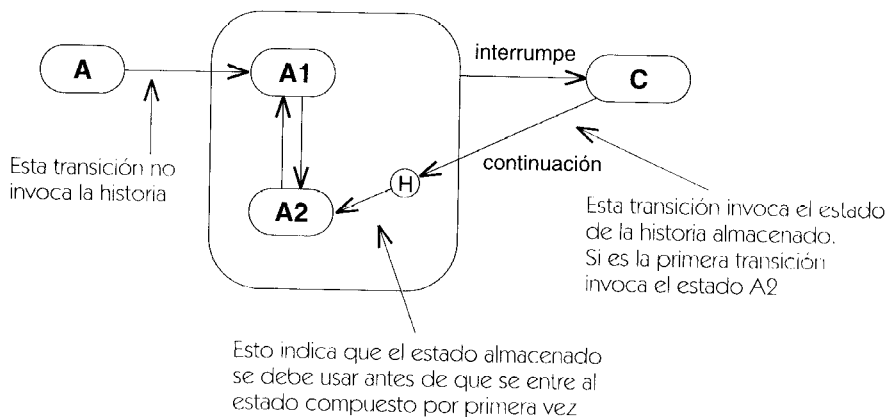


Figura 13.90 Estado de historia

estado de origen

Estado del que parte una transición dentro de la máquina de estados. La transición se le aplica al estado de origen. Si un cierto objeto está en el estado o un estado está anidado en su interior, entonces la transición es un candidato para el disparo.

estado de referencia a submáquina

Estado que hace referencia a una submáquina de estados. Una copia de esta submáquina forma parte implícita de la máquina de estados que la encierra, en el lugar del estado de referencia a submáquina. Puede contener estados abreviados externos, que identifican a estados de la submáquina.

Véase también estado, máquina de estados, estado abreviado externo.

Semántica

Los estados de referencia a submáquina equivalen a insertar una copia de la submáquina en lugar del estado de referencia.

Notación

Los estados de referencia a submáquina se dibujan como un símbolo de estado con una etiqueta de la forma

include nombre-submáquina

Se pueden dibujar flechas de transición hacia estados externos situados dentro del estado de referencia a submáquina. Se dibujan como transiciones abreviadas —esto es, flechas que terminan en una barra transversal—. La barra transversal se etiqueta con el nombre del estado dentro de la submáquina a que se hace referencia.

Ejemplo

La Figura 13.91 muestra parte de una máquina de estados que contiene un estado de referencia a submáquina. La máquina de estados que lo contiene vende entradas a aquellos clientes que tienen cuentas. Tiene que identificar al cliente como parte de su tarea. La identificación del cliente es un requisito de otra máquina de estados, así que se ha definido como otra máquina de estados por separado. La Figura 13.92 muestra la definición de la máquina de estados **Identificar**, que es empleada como submáquina por otras máquinas de estados. La entrada normal en la submáquina se encarga de leer la tarjeta del cliente, pero existe un estado de entrada específico que ofrece la introducción manual del nombre del cliente por parte del encargado de la taquilla. Si tiene éxito el proceso de identificación, la submáquina concluye en su estado final. En caso contrario, pasa al estado **Incorrecto**.

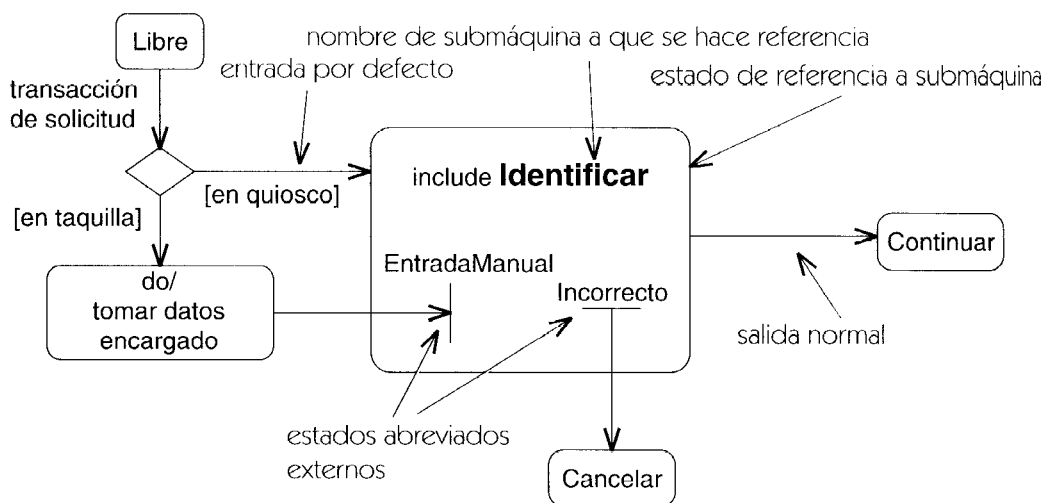


Figura 13.91 Estado de referencia a submáquina

En la Figura 13.91, la referencia a la submáquina se muestra mediante un icono de estado con la palabra reservada **include** y el nombre de la submáquina. La entrada normal a la submáquina se muestra mediante una flecha que llega a su periferia. Esta transición activa el estado inicial de la submáquina. La salida normal se muestra mediante una transición de finalización que sale de la periferia. Esta transición se dispara si la submáquina finaliza normalmente.

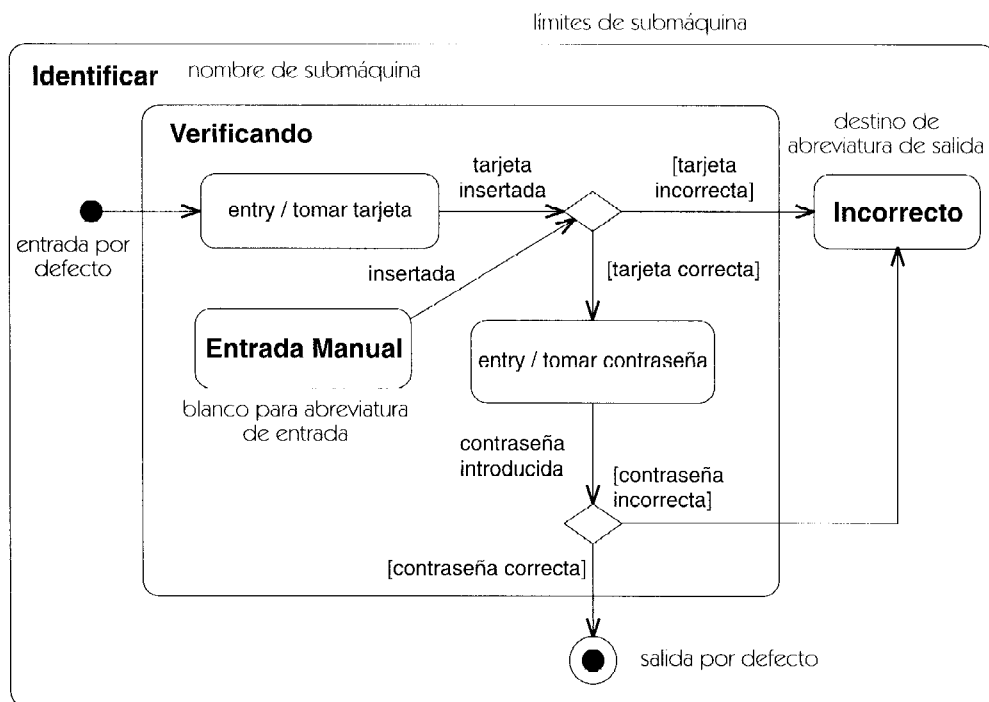


Figura 13.92 Definición de submáquina

La entrada al estado explícito **EntradaManual** se muestra mediante una transición a una abreviatura situada dentro del símbolo de referencia a la submáquina. La abreviatura está etiquetada con el nombre del estado destino de la submáquina. De forma análoga, la salida del estado explícito **Incorrecto** se muestra mediante una transición de finalización procedente de una abreviatura. Las transiciones a abreviaturas pueden tener o no un evento disparador.

estado de sincronización

Estado especial que permite la sincronización del control entre dos regiones concurrentes de una máquina de estados.

Véase también transición compleja, estado compuesto, bifurcación, unión, máquina de estados, transición.

Semántica

Un estado compuesto puede constar de varias regiones concurrentes, cada una de las cuales posee su propio conjunto secuencial de estados. Cuando se entra en un estado compuesto concurrente, todas las regiones concurrentes pasan a estar activas. En cada región concurrente hay un hilo de control y cada uno de ellos se ejecuta independientemente de los demás (esto es lo que significa la concurrencia). En algunas ocasiones, sin embargo, es preciso sincronizar el control entre regiones concurrentes. Un posible enfoque consiste en que una transición de una región tenga una condición de guarda que dependa del estado de otra región. Esto puede ser útil para la exclusión mutua, incluyendo los recursos compartidos, pero no captura aquellas situaciones en que la actividad de una región tenga consecuencias subsiguientes en otra (aun cuando la primera región pueda seguir adelante después). Para capturar esta última situación pueden utilizarse estados de sincronización.

Un estado de sincronización es un estado especial que conecta dos regiones concurrentes. Las regiones pueden ser pares, esto es, dos regiones concurrentes que pertenezcan a un mismo estado compuesto, o bien pueden estar anidadas entre pares, a cualquier profundidad. Estas regiones no pueden estar relacionadas secuencialmente.

Hay una transición que conecta la salida de una bifurcación situada en una región con la entrada del estado de sincronización y otra transición que conecta la salida del estado de sincronización con una unión de la otra región. En otras palabras, un estado de sincronización es una zona intermedia que conecta indirectamente una bifurcación de una región con una unión de otra región. La bifurcación y la unión tienen que tener ambas un estado de entrada y un estado de salida dentro de sus propias regiones.

El estado de sincronización recuerda el disparo de una transición en la primera región hasta que se dispara la transición de unión en la segunda región. Si las condiciones de la transición de unión se satisfacen antes de que esté activado el estado de sincronización, entonces la unión deberá esperar a que se dispare la transición de la primera región. En otras palabras, representa una transición de la segunda región que no puede dispararse mientras no se haya disparado una transición en la primera región. En caso contrario, se bloquea hasta que se dispara la primera transición. Obsérvese que dado que toda bifurcación y toda unión tienen que tener una entrada y una salida dentro de su propia región, los estados de sincronización no cambian

el comportamiento secuencial fundamental de cada una de las regiones concurrentes, ni tampoco alteran las reglas de anidamiento para formar estados compuestos (salvo que el estado de sincronización y sus arcos no pertenecen a ninguna de las regiones concurrentes sino más bien al estado compuesto por ambas regiones).

Las situaciones del tipo productor-consumidor son ejemplos típicos de los estados de sincronización. El productor dispara una transición que activa el estado de sincronización; el consumidor posee una transición que requiere el estado de sincronización para poder dispararse.

Si la transición de entrada al estado de sincronización forma parte de un bucle, es posible que la primera región puede adelantarse a la segunda. En otras palabras, el estado de sincronización puede tener más de un *símbolo* (para usar el término de las redes de Petri). Por tanto, un estado de sincronización, a diferencia de un estado normal, representa un contador o cola (esto último si la información fluye entre las regiones, si el estado de sincronización es un estado de flujo de objeto, por ejemplo). Por defecto, los estados de sincronización pueden contener un número ilimitado de símbolos, pero el creador del modelo puede especificar un límite superior para el número que puede contener el estado de sincronización. Si se excede la capacidad del estado de sincronización, se producirá un error en tiempo de ejecución. Lo más frecuente es que el límite sea ilimitado o uno, entonces esto último representa un simple interruptor. En general, un límite de uno sólo se utilizaría si fuera posible garantizar que no se va a producir un desbordamiento. Esto es responsabilidad del creador del modelo.

Si existen símbolos en un estado de sincronización cuando finaliza el estado compuesto que lo contiene, se destruyen. El estado de sincronización está vacío siempre que se entra en la región compuesta.

Puede haber más de un arco de entrada que penetre en un estado de sincronización, pero todos ellos tienen que provenir de bifurcaciones pertenecientes a la misma región secuencial. Análogamente, puede haber más de un arco de salida que salga de un estado de sincronización para llegar a uniones de la sección secuencial. Como todas las regiones son secuenciales, no hay peligro de conflictos entre múltiples arcos.

Un estado de sincronización puede ser un estado de flujo de objeto. En tal caso, representa una cola de valores que van pasando de una región a otra.

Notación

Los estados de sincronización se muestran como un circulito con un único límite superior en su interior, bien sea un entero o un asterisco (*) que denota un valor ilimitado. Hay una flecha de transición que parte desde un símbolo de barra de sincronización (una barra gruesa) situado en el estado de sincronización y otra flecha de transición que parte del estado de sincronización y llega hasta un símbolo de barra de sincronización que se encuentra en otra región (Figura 13.93).

Preferiblemente, el estado de sincronización se dibujará en el límite entre dos regiones, pero esto no siempre es posible (las regiones pueden no ser adyacentes) y en todo caso la topología de la conexión no es ambigua.

Dentro de un diagrama de actividades, cada arco de transición representa implícitamente un estado. Por tanto, se puede trazar un flecha desde la salida de una bifurcación hasta la entrada de una unión sin mostrar explícitamente el estado de sincronización (pero es necesario que el estado de sincronización muestre un límite explícito).

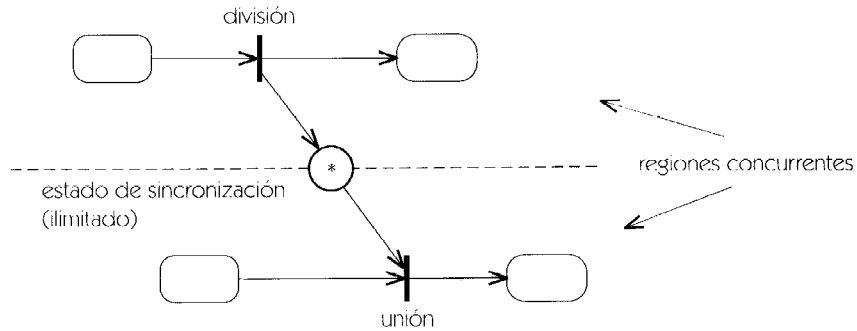


Figura 13.93 Configuración de un estado de sincronización

Ejemplo

La Figura 13.94 muestra un diagrama de estados correspondiente a una situación de compra de entradas. La emisión de entradas y el cobro de las mismas se producen concurrentemente, salvo que sea preciso seleccionar los asientos antes de que se puedan calcular y enviar los costes. Esta sincronización se muestra mediante la inserción de un estado de sincronización entre

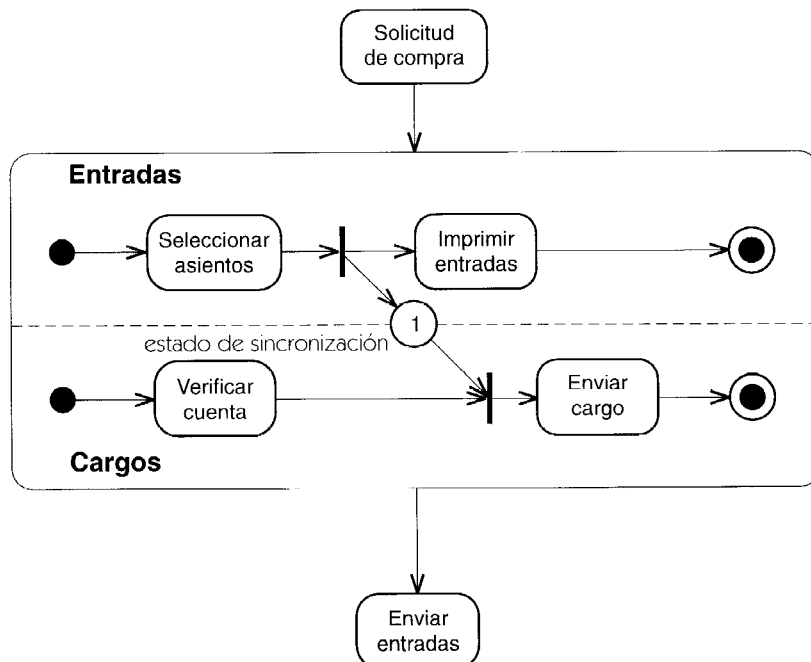


Figura 13.94 Estado de sincronización para pedido único

Seleccionar asientos y **Enviar cargo**. Existe una bifurcación después de **Seleccionar asientos**, porque va seguida tanto por **Imprimir entradas** como por el estado de sincronización. **Imprimir cargo** no tiene que esperar a la sincronización. Existe una unión antes de **Enviar cargo**, porque tiene que esperar tanto a **Verificar cuenta** como al estado de sincronización. Cuando han concluido tanto **Imprimir entradas** como **Enviar cargo**, el estado compuesto concluye y se ejecuta **Enviar entradas**.

El estado de sincronización tiene un límite superior de uno. No es necesario tener un límite mayor porque sólo hay una sincronización por cada ejecución del estado compuesto.

La Figura 13.95 muestra una versión de procesamiento por lotes de este proceso de llenado de pedidos. En esta variación, se pueden hacer pedidos sin estar conectado al sistema. Los pedidos los hace un servidor y los cargos los hace otro. No se puede hacer un cargo antes de haber llenado el pedido, pero se pueden hacer pedidos antes de hacerse los cargos así que el estado de sincronización tiene un límite superior ilimitado. Se trata de la situación clásica productor-consumidor.

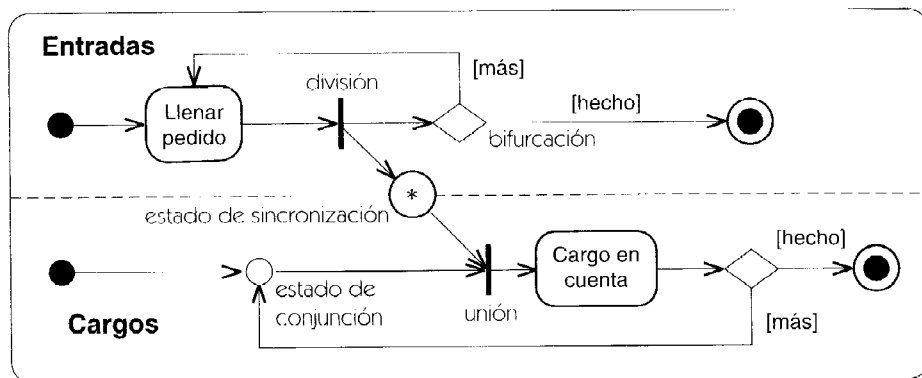


Figura 13.95 Situación productor-consumidor con estado de sincronización ilimitado

Discusión

Los estados de sincronización proporcionan la capacidad de modelar situaciones productor-consumidor con un coste mínimo y con mayor seguridad que otras estructuras de concurrencia más generales, porque cada región concurrente mantiene un único hilo de control en todo momento.

No hay peligro de desbordamiento (si el estado de sincronización es ilimitado) porque el estado de sincronización se vacía siempre que se sale del supraestado. Si se utilizan estados de sincronización dentro de bucles que contengan ramificaciones, existe el peligro de un bloqueo: esto sucederá si una región ha terminado pero la otra espera aun símbolo de sincronización que no va a llegar nunca. También es posible un interbloqueo si ambas regiones se encuentran en una ramificación que espera un símbolo procedente de otra región. No hay forma de evitar por completo estas situaciones en sistemas concurrentes que permitan la toma de decisiones. Aunque no haya estados de sincronización, no se puede garantizar la finalización como consecuencia del problema de la detención de la ejecución.

estado de unión o conjunción

Se trata de un pseudoestado que forma parte de una sola transición global en una máquina de estados. No interrumpe un paso simple del tipo “ejecutar hasta finalizar”, en la ejecución de una transición.

Véase también bifurcación, reunificación.

Semántica

Una transición de una máquina de estados puede cruzar los límites de varios estados compuestos desde el estado origen hasta el estado destino. Al ejecutar una de estas transiciones, pueden invocarse una o más acciones de entrada o de salida. En algunas ocasiones es necesario entrelazar una o más acciones de la transición con las acciones de entrada y de salida que están asociadas a los estados anidados. Eso no puede hacerse con una sola transición, que tiene asociada una sola acción.

También es conveniente permitir que varios disparadores posean un mismo resultado o permitir que un único disparador tenga varios resultados posibles con diferentes condiciones de guarda.

En este sentido, un estado de unión es un pseudoestado que hace posible construir una única transición global a partir de una serie de fragmentos de transición.

Un estado de unión puede poseer uno o más segmentos de transición entrantes y uno o más segmentos de transición salientes. No puede poseer una actividad interna, ni una submáquina, ni ninguna transición saliente con eventos disparadores. Se trata de un estado neutro que sirve para estructurar las transiciones y no de un estado que pueda estar activo durante un tiempo finito.

Los estados de unión se emplean para estructurar una transición procedente de varios segmentos. Sólo el primer segmento de una cadena de estados de unión podrá tener un evento disparador, pero todos ellos pueden tener condiciones de guarda. Los segmentos subsiguientes deben carecer de disparadores. La condición de guarda efectiva es la conjunción de todas las condiciones de guarda originales. No se dispara la transición mientras no se cumpla todo el conjunto completo de condiciones. En otras palabras, la máquina de estados no puede permanecer en el estado de unión.

Si hay múltiples transiciones que entran en un único estado de unión, cada una puede tener un disparador diferente, o bien puede carecer de él. Cada una de las rutas que atraviesan un conjunto de estados de unión representa una transición diferente.

Una transición saliente puede tener una condición de guarda. Si existen múltiples transiciones salientes, cada una de ellas debe poseer una condición de guarda diferente. Esto es una bifurcación.

Una transición saliente puede tener asociada una acción. (El estado de unión puede poseer una acción interna, pero esto equivale a asociar una acción a la transición saliente, que es el enfoque preferible.) La acción se ejecuta siempre y cuando se cumplan todas las condiciones de

guarda, incluyendo aquellas que se encuentren en los segmentos subsiguientes. Una transición no puede efectuar un “disparo parcial” de tal modo que se detenga el estado de unión. Tiene que alcanzar un estado normal.

Cuando se dispara una transición entrante, la transición saliente se dispara inmediatamente. Entonces se ejecuta cualquier acción que pudiera estar asociada. La ejecución de la transición entrante y de la transición saliente forma parte de un único paso atómico (un paso de los que se ejecutan hasta finalizar), esto es, no puede ser interrumpida por un evento ni por otras acciones.

Notación

Los estados de unión se muestran en la máquina de estados mediante un circulito. No tienen nombre. Pueden tener flechas de transición entrantes y salientes.

Ejemplo

La Figura 13.96 muestra dos transiciones completas del estado **S** al estado **T** —una transición de un solo segmento disparada por el evento **f** y una transición de múltiples segmentos disparada por el evento **e**, que se ha estructurado empleando dos estados de unión—. Las anotaciones muestran el entrelazado de las acciones de transición con las acciones de entrada y salida.

Observe que la situación del rótulo de acción en la línea de transición no posee significado propio. Si la acción **d** hubiese estado situada dentro del estado **X**, se habría ejecutado en todo caso después de salir del estado **X** y antes de entrar en el estado **Y**. Por tanto, debe dibujarse en la posición más externa de la transición.

Para otros ejemplos, véanse la Figura 13.95 y la Figura 13.180.

Véanse también los iconos de control para otros símbolos abreviados que pueden incluirse en los diagramas de estados y en los diagramas de actividades

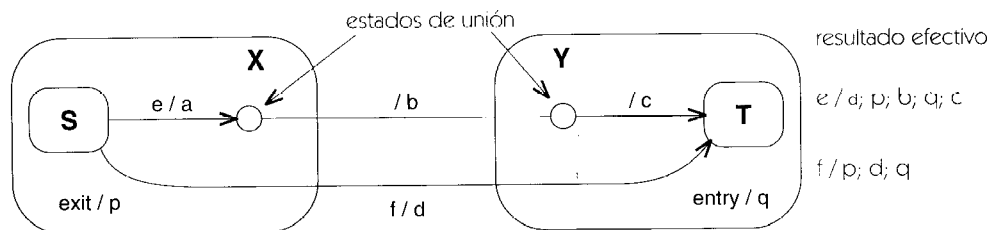


Figura 13.96 Estados de unión

estado final

Un estado especial dentro de un estado compuesto que, cuando está activo, indica que la ejecución del estado compuesto ha terminado. Cuando un estado compuesto alcanza su estado fi-

nal, se dispara una transición de terminación, que sale del estado compuesto y se puede activar si se satisface su condición de guarda.

Véase también actividad, transición de finalización, destrucción.

Semántica

Para promover la encapsulación, es deseable separar la vista exterior de un estado compuesto de los detalles internos tanto como sea posible. Desde el exterior, el estado se ve como entidad opaca con una estructura interna que se oculta. Desde el punto de vista exterior, las transiciones van a y desde el propio estado. Desde un punto de vista interior, conectan con los subestados dentro del estado. Los estados inicial y final son mecanismos para permitir la encapsulación de estados.

Un estado final es un estado especial que indica que la actividad del estado compuesto está completa y que una transición de finalización que sale del estado compuesto está activada. Un estado final no es un pseudoeestado. Un estado final puede ser activado por un período de tiempo, a diferencia de un estado inicial que transita inmediatamente a su sucesor. El control puede permanecer dentro de un estado final mientras se espera la terminación de otros subestados concurrentes del estado compuesto —es decir, mientras espera para sincronizarse con múltiples hilos de control para unirse—. No obstante, las transiciones salientes disparadas por eventos no están permitidas en un estado final (de lo contrario, sería justo un estado normal). Un estado final puede tener cualquier número de transiciones entrantes dentro del estado compuesto que incluye, pero ninguna transición hacia afuera del estado que incluye. Las transiciones entrantes son transiciones normales y pueden tener el conjunto completo de disparadores, condiciones de guarda, y acciones.

Si un objeto alcanza su estado final de nivel superior, la máquina de estados termina y se destruye el objeto.

Notación

Un estado final se representa como un icono de un ojo de buey —es decir, un pequeño disco negro relleno rodeado por una pequeña circunferencia—. Se coloca dentro del estado compuesto que lo contiene, cuya terminación representa (Figura 13.97). Solamente puede ocurrir un estado final (directamente) dentro de un estado compuesto. Los estados finales adicionales pueden

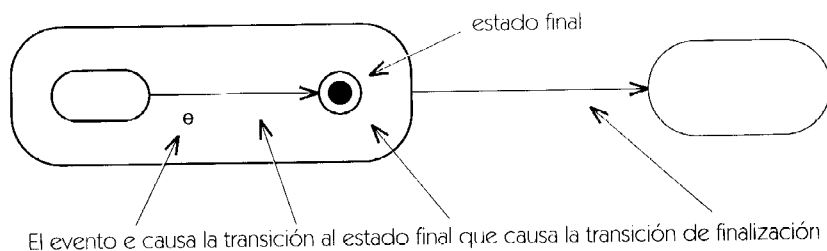


Figura 13.97 Estado final

ocurrir, sin embargo, dentro de estados compuestos anidados. Por conveniencia, el símbolo final del estado se puede repetir dentro de un estado, pero cada copia representa el mismo estado final.

estado inicial

Se trata de un pseudoestado que indica el punto de partida por defecto para una transición cuyo destino es el límite de un estado compuesto.

Véase también estado compuesto, creación, acción de entrada, iniciación, estado de unión.

Semántica

Para promover el encapsulamiento, resulta deseable separar tanto como sea posible la vista externa de un estado compuesto y los detalles internos. Desde el exterior, se ve el estado como una entidad opaca con una estructura interna oculta. Desde el punto de vista externo, las transiciones van y vienen a ese estado. Desde el punto de vista interno, están conectadas a subestados propios de ese estado.

El nombre de estado inicial alude a un estado neutro (un pseudoestado) que representa un punto de conexión para una transición entrante que llega a los límites del estado compuesto. No se trata de un estado real; el control no puede permanecer en su interior. Se trata, más bien, de una forma sintáctica de indicar dónde debería ir el control. Todo estado inicial debe tener una transición saliente sin disparador (una transición sin evento disparador, que por tanto se activará automáticamente en cuanto se entre en el estado inicial). La transición de finalización establece una conexión con un estado real del estado compuesto. La transición de finalización puede contener una acción. La acción se ejecuta cuando se entra en el estado después de ejecutar la acción de entrada (si existe) del estado compuesto. Esto permite asociar una acción a la entrada por defecto, además de la acción de entrada (que se ejecuta para todas las entradas, tanto si son por defecto como si no). Esta acción puede acceder al evento actual implícito, esto es, al evento que haya disparado el primer segmento de la transición que en última instancia ha dado lugar a la transición al estado inicial.

Un estado inicial no puede poseer una transición saliente con un evento disparador. Una transición entrante equivale a una transición entrante al estado compuesto envolvente y debe evitarse. Estas transiciones deben conectarse al compuesto.

Con gran frecuencia, la transición al estado inicial no tiene guarda. En tal caso, tiene que ser la única transición procedente del estado inicial. Se puede proporcionar un conjunto de transiciones salientes con condiciones de guarda, pero estas condiciones de guarda deben abarcar absolutamente todos los casos posibles (o bien, más sencillamente, una de ellas puede tener la condición de guarda **else**). Lo importante es que el control debe salir inmediatamente del estado inicial. No se trata de un estado real, así que debe dispararse alguna transición.

El estado inicial del estado de nivel más alto de una clase representa la creación de una nueva instancia de la clase. Cuando se sigue la transición saliente, el evento actual implícito es el evento de creación del objeto y posee los argumentos que se le pasan por parte de la operación del constructor. Estos valores están disponibles dentro de las acciones de la transición saliente.

Creación del objeto

El estado inicial del estado compuesto de nivel más elevado de una clase es ligeramente distinto. Puede poseer un disparador que tenga el estereotipo «**create**», junto con un evento disparador con un nombre y unos parámetros. Quizá haya múltiples transiciones de esta clase con distintos disparadores. La signature de cada disparador debe coincidir con una operación de creación de la clase. Cuando se instancia un nuevo objeto de la clase, se dispara la transición que corresponde a su operación de creación; esa transición recibe los argumentos de la llamada a la operación de creación.

Notación

El estado inicial se representa mediante un circulito negro situado dentro del símbolo de su estado compuesto. Se pueden conectar a este símbolo las flechas de transición salientes. Sólo puede haber un estado inicial (directamente) dentro de cada estado compuesto. Sin embargo, puede haber estados iniciales adicionales dentro de estados compuestos anidados.

Ejemplo

En la Figura 13.98 comenzamos en el estado **X**. Cuando se produce un evento, se dispara la transición y se ejecuta una acción. La transición pasa al estado **Y**. Se ejecuta la acción de entrada **b**, y el estado inicial queda activado. La transición saliente se dispara inmediatamente, ejecutando la acción **c** y pasando al estado **Z**.

Por otra parte, si el evento **f** se produce cuando el sistema se encuentra en el estado **X**, entonces se dispara la otra transición y se ejecuta la acción **d**. Esta transición pasa directamente al estado **Z**. El estado inicial no está implicado. Dado que el control pasa al estado **Y**, se ejecuta la acción **b** pero la acción **c** no llega a ejecutarse en este caso.

En la Figura 13.99, el estado inicial posee una bifurcación. Suponga una vez más que el sistema comienza en el estado **X**. Cuando se produce un evento, el sistema pasa al estado **Y** y se ejecuta la acción de entrada **b**. El control pasa al estado inicial. Se comprueba atributo de tamaño del objeto propietario. Si es 0, el control pasa a **Z**; en caso contrario, el control pasa al estado **W**.

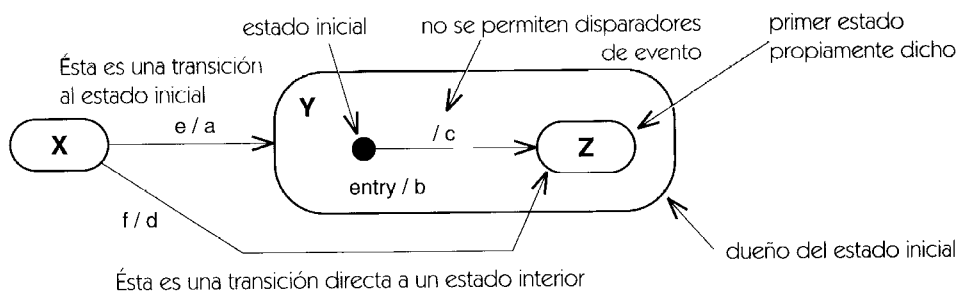


Figura 13.98 Estado inicial

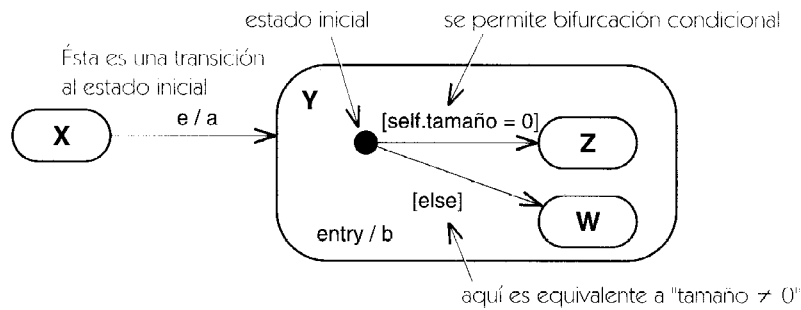


Figura 13.99 Estado inicial con ramificación

estado simple

Estado que no contiene otros estados anidados en su interior. Un conjunto de estados anidados forma un árbol y los estados simples son las hojas. Los estados simples no poseen una subestructura. Pueden tener transiciones internas, acciones de entrada y acciones de salida. Por contrastar con: estado compuesto.

estereotipo

Nueva clase de elemento del modelo definida dentro del modelo y basada en alguna clase existente de elementos del modelo. Los sistemas pueden extender la semántica pero no la estructura de clases preexistentes del metamodelo.

Véase también restricción, valor etiquetado.

Véase el Capítulo 14, Elementos Estándar, para consultar una lista de estereotipos predefinidos.

Semántica

Los estereotipos representan una variación de algún elemento existente del modelo, con la misma forma (tal como los atributos y relaciones) pero con una intención diferente. En general, los estereotipos representan una distinción de utilización. Un elemento con estereotipo puede tener restricciones adicionales, más allá de las que posea el elemento base, así como otro aspecto visual diferente. Se supone que los generadores de código y demás herramientas tratarán de forma especial a los elementos con estereotipos, generando por ejemplo un código diferente. La intención es que una herramienta genérica de modelado, como un editor o repositorio de modelos, trate (para casi todos los propósitos) a los elementos con estereotipos como elementos ordinarios con alguna información textual adicional, al mismo tiempo que se distingue ese elemento para ciertas operaciones semánticas tales como la comprobación de estar bien formados, la generación de código y la creación de informes. Los estereotipos representan uno de los mecanismos de extensión que se han incorporado a UML.

Todo estereotipo se deriva de alguna clase base de elemento del modelo. Todos los elementos que lleven el estereotipo tendrán las propiedades de la clase base de ese elemento del modelo.

Los estereotipos también se pueden especializar a partir de otros estereotipos. Las definiciones de estereotipos son elementos generalizables. El estereotipo hijo tiene las propiedades del estereotipo padre. En última instancia, todo estereotipo está basado en alguna clase de elemento del modelo.

Los estereotipos pueden poseer una lista de marcadores requeridos y algunos de ellos pueden tener un valor por defecto que se utilizará si no se proporciona un valor etiquetado explícito. El intervalo de valores permitidos para cada marcador también se puede especificar. Todo elemento que lleve el estereotipo debe tener valores etiquetados con los marcadores que se enumeran. Los marcadores que posean valores por defecto quedarán implícitos automáticamente si no son explícitos en aquellos elementos que lleven el estereotipo.

Los estereotipos pueden tener una lista de restricciones que añadirán condiciones a las implicadas por el elemento base. Todas las restricciones serán de aplicación a todo aquel elemento del modelo que lleve el estereotipo. Además, los elementos del modelo están sometidos a las restricciones aplicables al elemento base.

Un estereotipo es una clase virtual del metamodelo (esto es, no se manifiesta en el metamodelo) que se añade dentro del modelo en lugar de modificar el metamodelo predefinido de UML. Por esta razón, los nombres de los estereotipos nuevos tienen que ser distintos de los nombres de metaclases existentes en UML y de los nombres de los demás estereotipos o palabras reservadas.

Todo elemento del modelo puede tener como máximo un estereotipo. Esta regla puede no ser esencial lógicamente pero simplifica la semántica y la notación de los estereotipos sin que haya una pérdida real de potencia, porque se permite la herencia múltiple de estereotipos. Los estereotipos pueden ser hijos de otros estereotipos. Toda situación en que un elemento tenga múltiples estereotipos se puede transformar en otra en que haya un solo estereotipo que sea hijo de otros. En algunas ocasiones, esto puede obligar a quien crea el modelo a combinar otros estereotipos, pero estimamos que la mayor sencillez del caso medio compensa esta incomodidad.

Hay ciertos estereotipos que se han predefinido en UML; otros pueden ser definidos por el usuario. Los estereotipos son uno de los tres mecanismos de extensión que existen en UML.

Véase restricción, valor etiquetado.

Véase el Capítulo 13, Enciclopedia de Términos, para consultar una lista de los estereotipos predefinidos.

Notación

La notación general para el uso de estereotipos consiste en emplear el símbolo del elemento base añadiendo una cadena que contiene una palabra reservada por encima del nombre del elemento base (si existe). La palabra reservada es el nombre del estereotipo entre comillas, que son el signo de cita que se emplea en Francés y también en otros idiomas, por ejemplo, «**entre comillas**». (Observe que las comillas se parecen a dobles corchetes angulares, pero se trata de un único carácter que se halla en la mayoría de las fuentes extendidas. Casi todos los computadores tienen una utilidad de mapa de caracteres en la que se pueden buscar símbolos especiales. Quienes tengan problemas tipográficos pueden utilizar dobles corchetes angulares.) La cadena que contiene la palabra reservada suele situarse encima o delante del elemento de

modelo que se está describiendo, también se puede utilizar esa palabra reservada como elemento de una lista. En tal caso, se aplica a los elementos subsiguientes de la lista hasta que la sustituya otra cadena de estereotipo, o bien hasta que aparezca una cadena vacía de estereotipo («») que ponga fin a su alcance. Observe que el nombre de un estereotipo no debe ser idéntico al de una palabra reservada predefinida que pueda ser aplicable al mismo tipo de elemento. (Para evitar confusiones, los nombres de palabras reservadas deben evitarse en todos los estereotipos, aunque sólo se puedan aplicar a otros elementos y sean distinguibles en principio.)

Para permitir una extensión gráfica limitada de la notación de UML, se puede asociar un icono o marcador gráfico (como una textura o color) a las estructuras.

UML no especifica la forma de especificación gráfica pero existen muchos formatos de mapa de bits y de trazos que se pueden utilizar en un editor gráfico (aunque su transportabilidad es un problema difícil). Se pueden utilizar los iconos de dos maneras. En un caso, pueden emplearse en lugar de la palabra reservada de estereotipo o además de ella, dentro del símbolo del elemento base del modelo en que se basa el estereotipo. Por ejemplo, en un rectángulo de clase el icono se sitúa en la esquina superior derecha del compartimento de nombre. De esta manera, se puede ver el contenido normal del elemento en su símbolo. Alternativamente, se puede “concentrar” todo el símbolo del elemento en un icono que contiene el nombre del elemento o que tiene el nombre por encima o por debajo de su icono. Se suprime el resto de la información contenida en el símbolo del elemento base del modelo.

La Figura 13.100 muestra diferentes formas de dibujar un clase con estereotipo.

UML evita el uso de marcadores gráficos tales como el color, que supone una dificultad para ciertas personas (daltónicos) y para tipos importantes de periféricos (como impresoras, copadoras y máquinas de fax). Ninguno de los símbolos de UML requiere el uso de estos marcadores gráficos. Los usuarios pueden emplear libremente los marcadores gráficos para sus propios propósitos (tales como resaltar elementos dentro de una herramienta) pero deben ser conscientes de sus limitaciones para el intercambio y tienen que estar dispuestos a utilizar las formas canónicas cuando sea necesario.

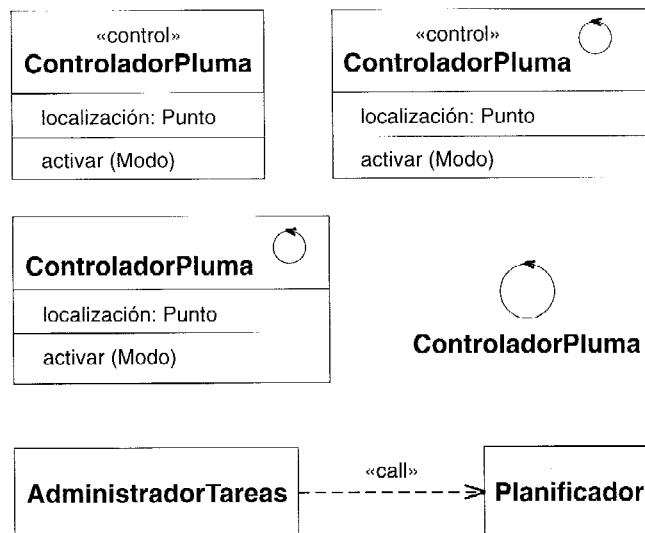


Figura 13.100 Variedades de notaciones de estereotipos

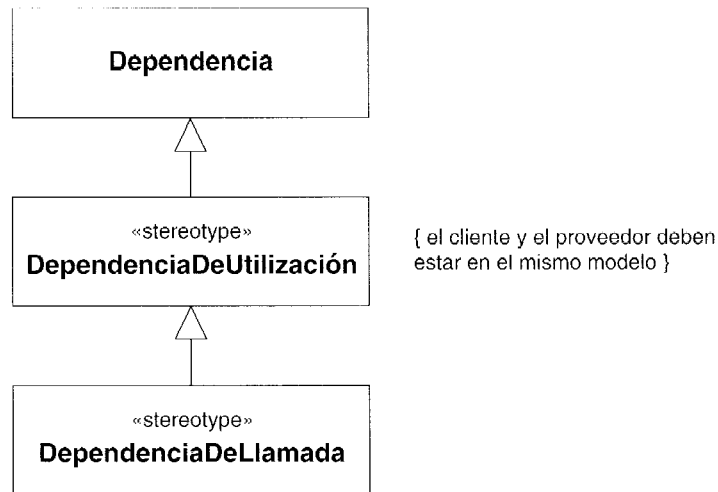


Figura 13.101 Declaración de un estereotipo

Declaración de estereotipos. La jerarquía de clasificación de los propios estereotipos se puede visualizar en un diagrama de clases. Sin embargo, se trata de un diagrama de metamodelo y debe ser distinguido (tanto por los usuarios como por las herramientas) de los diagramas ordinarios del modelo. En tales diagramas, cada estereotipo se representa mediante un símbolo de clase (un rectángulo) con la palabra reservada «**stereotype**» (estereotipo). Las relaciones de generalización pueden mostrar la jerarquía extendida del metamodelo (Figura 13.101). Dado el peligro que supone extender la jerarquía interna del metamodelo, las herramientas pueden exponer esta capacidad en los diagramas de clases, pero pueden no hacerlo. La declaración de los nombres de los estereotipos no es una capacidad que requieran los constructores de modelos ordinarios, pero se puede efectuar como tarea de apoyo.

etiqueta

El término que denota el uso de una cadena en un diagrama. Se trata de un término puramente de notación.

Véase también diagrama.

Notación

Una etiqueta es una cadena gráfica que está asociada lógicamente a otro símbolo de un diagrama. Visualmente, la asociación suele ser cuestión de situar la cadena en una región cerrada o poner la cadena cerca del símbolo. Para algunos símbolos la cadena se coloca en una posición determinada (tal como la parte inferior de una línea) pero para la mayoría de los símbolos la cadena tiene que estar “cerca” de una línea o de un icono. Una herramienta de edición puede mantener un enlace gráfico interno explícito entre la etiqueta y un símbolo gráfico para que la etiqueta quede conectada lógicamente con el símbolo aun cuando ambos estén separados visualmente. Pero el aspecto final del diagrama es una cuestión de juicio estético y

debería hacerse de tal modo que no haya confusión acerca del símbolo al que está asociada la etiqueta. Aun cuando la asociación pueda no ser evidente a partir de una inspección visual del diagrama, la asociación será clara y sin ambigüedad en el valor de estructura gráfica (y por tanto no implicará ambigüedad en la correspondencia semántica). Una herramienta puede mostrar visualmente la asociación de una etiqueta a otro símbolo empleando diferentes ayudas (tales como una línea coloreada, o el parpadeo de elementos asociados) para mayor comodidad.

evento

La especificación de una ocurrencia destacada que tiene una localización en el tiempo y en el espacio.

Véase también máquina de estados, transición, disparador.

Semántica

Dentro de una máquina de estados, una ocurrencia de un evento puede disparar una transición de estado. Un evento tiene una lista de parámetros (posiblemente vacía) que transportan la información desde el creador del evento a su receptor. El tiempo en que ocurre el evento es un parámetro implícito de cualquier evento. Otros parámetros son parte de la definición de un evento.

Una ocurrencia (instancia) de un evento tiene argumentos (valor real) que se corresponden a cada parámetro del evento. El valor de cada argumento está disponible para una acción unida a una transición accionada por el evento.

Hay cuatro clases de eventos:

- | | |
|-------------------|--|
| evento de llamada | La recepción de una petición para invocar una operación. El resultado previsto es la ejecución de la operación accionando una transición en el receptor. Los parámetros del evento son una referencia a la operación y a los parámetros de la operación e, implícitamente, un puntero de vuelta. El llamador recupera control cuando la transición es completa (o inmediatamente si no se dispara ninguna transición). |
| evento de cambio | La satisfacción de una condición booleana especificada por una expresión en el evento. No hay parámetros. Este tipo de evento implica una prueba continua de la condición. El evento ocurre cuando la condición cambia de falso a verdadero. En la práctica, sin embargo, los tiempos en los cuales la condición puede ser satisfecha se pueden restringir a menudo a la ocurrencia de otros eventos, para no requerir comprobación constante. |
| evento de señal | La recepción de una señal, la cual es una entidad a la que se ha dado nombre explícitamente, prevista para la comunicación explícita entre objetos. Una señal tiene una lista explícita de parámetros. Es enviada explícitamente por un objeto a otro objeto o conjunto de objetos. Una difusión general de un evento se puede ver como el envío |

de una señal al conjunto de todos los objetos, aunque en la práctica, se puede implementar de forma diferente, por razones de eficiencia.

El remitente especifica explícitamente los argumentos de la señal cuando se envía. Una señal enviada a un conjunto de objetos puede accionar cero o una transición en cada uno de ellos.

Las señales son los medios explícitos por los cuales los objetos pueden comunicarse entre sí de forma asíncrona. Para realizar la comunicación síncrona, se deben utilizar dos señales asincrónicas, una en cada dirección de la comunicación.

Las señales son generalizables. Una señal del hijo se deriva de una señal del padre; hereda los parámetros del padre y puede agregar parámetros adicionales de sí mismo. Una señal del hijo satisface un disparador lo que requiere a uno de sus antecesores.

evento de tiempo

La satisfacción de una expresión de tiempo, tal como la ocurrencia de un tiempo absoluto o del paso de una cantidad de tiempo dada después de que un objeto entre en un estado. Observe que el tiempo absoluto y el tiempo transcurrido se pueden definir con respecto a un reloj del mundo real o a un reloj interno virtual (en cuyo caso, puede diferir para diversos objetos).

Notación

Véase el tipo específico de evento para los detalles sobre la notación.

Elementos estándar

create, destroy.

evento actual

El evento que accionó la ejecución de una etapa atómica en la máquina de estados.

Véase también ejecución atómica, máquina de estados, transición.

Semántica

Por conveniencia, una máquina de estados puede atravesar varios segmentos de transición en respuesta a un evento. Todos, excepto el segmento final de la transición, pasan a pseudoestados —es decir, estados ficticios cuyo propósito es ayudar a estructurar la máquina de estados, pero que no esperan eventos exteriores. En principio, sería posible reunir todos los segmentos en una transición, pero la separación en múltiples segmentos usando pseudoestados permite compartir las subsecuencias comunes entre varias transiciones.

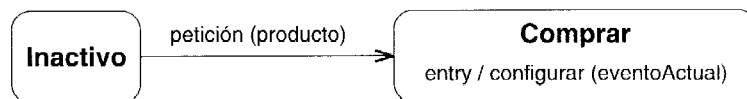


Figura 13.102 Uso del evento actual

La ejecución de una secuencia de segmentos de transición es atómica —es decir, es parte de un solo paso de ejecución que no puede interrumpir un evento exterior—. Durante la ejecución de tal secuencia de transiciones, las acciones y las condiciones de guarda unidas a los segmentos tienen acceso implícito al evento que accionó la primera transición y a los parámetros de ese evento. Este evento se conoce como evento actual durante una transición. El tipo del suceso actual se puede discriminar por una operación polimórfica o una sentencia selectiva múltiple. Después de saber el tipo exacto, será posible acceder a sus parámetros.

El evento actual es particularmente útil en la transición inicial de un nuevo objeto para obtener los parámetros de la creación. Cuando se crea un nuevo objeto, el evento que lo crea se convierte en el evento actual y sus parámetros están disponibles durante la transición inicial de la nueva máquina de estados del objeto.

Ejemplo

La Figura 13.102 muestra una transición del estado **Inactivo** al estado de **Comprar** accionado por el evento **petición**. La acción de entrada de **Comprar** llama a la operación de **configurar**, que utiliza el evento actual. El programa puede tener acceso al evento actual para obtener el evento **petición** y su parámetro, **producto**. Si hay posibles múltiples enlaces al evento actual, el programa puede necesitar que una sentencia case para obtener el evento correcto del disparador. La sintaxis es específica del lenguaje de programación.

Notación

El evento actual se puede señalar en una expresión por la palabra clave **currentEvent**. Una expresión en un lenguaje en particular puede proporcionar una sintaxis más detallada.

evento de cambio

Evento de una expresión booleana que se satisface gracias a un cambio en uno o más valores de aquellos a los que hace referencia.

Véase también condición de guarda.

Semántica

Un evento de cambio contiene una condición especificada por una expresión booleana. El evento no tiene parámetros. Ocurre cuando la condición tiene el valor verdadero tras haber tenido el valor falso, debido a un cambio en una o más variables de las que depende la condición.